

[OMNISTREAM_MASTER_GUARDIAN]
[DOCUMENT_TYPE=PRODUCT_CONSTITUTION+VISION_GUARDIAN]
[AUTHORITY=HIGH]
[PRIORITY=PRODUCT_INTENT>FEATURE_COUNT>VISUAL_NOVELTY]
[STATUS=FOUNDATIONAL]

PURPOSE:

This document is the permanent guardian of the original OMNISTREAM motive, vision, and product boundaries. Any future developer, AI agent, model, refactor, feature, automation, or architectural decision MUST consult this document before making changes that could alter the product's identity, core experience, scope, privacy, security, or free-first philosophy.

OMNISTREAM is a personal media environment combining two deliberately different experiences:

U-TUBE:

A convenient personal interface for discovering and watching supported online video content.

CINEMORPH:

A cinematic/theater presentation system designed primarily for larger screens that transforms supported media into an immersive theatrical viewing experience.

OMNISTREAM IS NOT:

A YouTube replacement.
A commercial streaming service.
A piracy platform.
An advertisement business.
A fake AI showcase.
A social-media clone.
A data-harvesting platform.
A provider-security circumvention system.
A guaranteed universal media decoder.
A guaranteed universal download system.
A claim that ordinary hardware becomes certified cinema hardware.

=====

CORE MOTIVE

=====

The original motivation is convenience.

The user wants a personal environment where supported online videos can be discovered and watched without repeatedly leaving the application, while maintaining a simple user-controlled experience.

The second motivation is cinematic presentation:

The user wants supported local/remote media to feel more like a theatrical presentation, with intelligent framing, theater presentation, screen geometry, curtains, ticketing, cinematic audio where technically possible, and an immersive viewing environment.

The project exists to improve the EXPERIENCE of consuming media.

It does not exist to reproduce every feature of a major commercial platform.

=====
PRIMARY PRODUCT PHILOSOPHY
=====

SIMPLICITY > COMPLEXITY
RELIABILITY > NOVELTY
TRUTH > MARKETING
USER EXPERIENCE > TECHNICAL SHOWCASE
PRIVACY > UNNECESSARY COLLECTION
LOCAL PROCESSING > UNNECESSARY CLOUD PROCESSING
FREE-FIRST > PAID DEPENDENCY
REAL FUNCTIONALITY > DECORATIVE FUNCTIONALITY
GRACEFUL FALLBACK > FAILURE
MAINTAINABILITY > ARCHITECTURAL THEATER

=====
U-TUBE GUARDIAN
=====

U-Tube must preserve the mental model:

SEARCH → SELECT → WATCH → DISCOVER → RETURN.

The primary source of discovery is supported online video search.

Secondary source:
locally stored subscriptions.

Additional discovery:
lightweight recommendations derived from recent activity.

Subscriptions must remain important.

History must preserve continuity.

Collections must remain simple.

The interface should be familiar to users of modern video platforms without attempting to become a pixel-perfect copy of another company's product or branding.

=====
U-TUBE NON-NEGOTIABLES
=====

SEARCH MUST BE REAL.

RESULTS MUST BE REAL.

VIDEO IDENTIFIERS MUST BE VALIDATED.

THUMBNAILS MUST NOT BE FABRICATED.

VIDEO TITLES MUST NOT BE FABRICATED.

CHANNEL INFORMATION MUST NOT BE FABRICATED.

HISTORY MUST REPRESENT ACTUAL PLAYBACK.

SUBSCRIPTIONS MUST REPRESENT ACTUAL USER ACTIONS.

RECOMMENDATIONS MUST BE DERIVED FROM REAL AVAILABLE DATA.

NO FAKE FALLBACK CONTENT.

NO FAKE API SUCCESS.

NO FAKE PLAYBACK.

NO DEAD PRIMARY BUTTONS.

NO UNNECESSARY USER CONFIGURATION.

=====
CINEMORPH GUARDIAN
=====

CineMorph must preserve the mental model:

CHOOSE MEDIA → TICKET → THEATER → SCREEN → WATCH → RESUME.

CineMorph is primarily local-media focused.

Supported remote media may be accepted where technically and legitimately supported.

Theater presentation must never become more important than the movie itself.

=====

CINEMORPH NON-NEGOTIABLES

=====

LOCAL FILE PLAYBACK MUST REMAIN LOCAL WHEN THE IMPLEMENTATION CLAIMS LOCAL PROCESSING.

THE ORIGINAL MEDIA MUST REMAIN AVAILABLE AS A SAFE BASELINE.

SMART FRAMING MUST NEVER BE REQUIRED FOR BASIC PLAYBACK.

AI FAILURE MUST NOT STOP THE MOVIE.

AUDIO ENHANCEMENT FAILURE MUST FALL BACK TO ORIGINAL AUDIO.

THEATER EFFECT FAILURE MUST NOT STOP PLAYBACK.

UNSUPPORTED CODECS MUST BE REPORTED HONESTLY.

MISSING AUDIO/SUBTITLE TRACKS MUST NOT BE FABRICATED.

1.43:1 PRESENTATION MUST NOT CLAIM TO RESTORE IMAGE INFORMATION THAT DOES NOT EXIST IN THE SOURCE.

=====

CINEMATIC EXPERIENCE GUARDIAN

=====

The intended theatrical experience includes:

ticket printing animation
ticket information
theater environment
seating perspective
cinema screen

curtains
screen opening
cinematic intro where applicable
1.90:1 mode
1.43:1 mode
Original mode
4:3 original presentation
smart framing
smooth screen-relative movement
optional audio enhancement
audio-track selection
subtitle selection
session persistence
ticket-based resume.

These elements are enhancements.

THE MOVIE ALWAYS HAS PRIORITY.

=====
SMART FRAMING GUARDIAN
=====

Smart framing may combine:

face detection
object detection
tracking
saliency
scene detection
composition analysis
geometry
cinematography heuristics
temporal smoothing
candidate generation
candidate scoring
safe zones
subtitle protection
text protection
cut detection.

No individual model should directly control the final presentation without safeguards.

The system MUST avoid:

jitter
rapid oscillation
unnecessary cropping
important-subject loss
subtitle clipping
important-text clipping
unnatural movement
excessive zoom
continuous micro-adjustments.

When confidence is low:

DO LESS.

When uncertain:

PREFER SAFE FRAMING.

=====

AI GUARDIAN

=====

AI exists to improve the product.

AI does not exist to make the product appear advanced.

Never add an LLM,vision model,or custom model solely because it sounds impressive.

Before adding a model ask:

WHAT EXACT PROBLEM DOES IT SOLVE?

CAN A DETERMINISTIC METHOD SOLVE IT?

CAN A SMALLER MODEL SOLVE IT?

DOES IT IMPROVE MEASURABLE USER EXPERIENCE?

WHAT IS THE PERFORMANCE COST?

WHAT IS THE FAILURE MODE?

WHAT IS THE FALLBACK?

If these questions cannot be answered:

DO NOT ADD THE MODEL.

=====

MODEL HIERARCHY

=====

Prefer:

RULES

→SPECIALIST MODEL

→HYBRID MODEL SYSTEM

→OPTIONAL SEMANTIC MODEL/LLM.

Do not use an LLM for frame-by-frame control.

Do not use an expensive model when a small specialist model provides sufficient quality.

=====

PERFORMANCE GUARDIAN

=====

Playback stability is more important than AI quality.

Priority:

VIDEO DECODING

→AUDIO SYNC

→RENDERING

→USER INPUT

→THEATER

→AI

→RECOMMENDATIONS

→DECORATIVE EFFECTS.

If the system becomes overloaded:

REDUCE AI.

REDUCE EFFECTS.

REDUCE ANALYSIS FREQUENCY.

KEEP PLAYBACK STABLE.

=====

SECURITY GUARDIAN

=====

Treat all external content as untrusted.

Validate:

URLs
video IDs
metadata
provider responses
local files
subtitles
model assets
external scripts.

Never expose secrets.

Never embed arbitrary URLs.

Never execute untrusted media content as code.

Never disable browser security merely to make development easier.

Never bypass DRM,provider access controls,authentication protections,or embedding restrictions.

=====

PRIVACY GUARDIAN

=====

Default preference:

LOCAL DATA.

History:
local.

Subscriptions:
local.

Collections:
local.

Tickets:
local.

Preferences:
local.

Local media:
local unless the user explicitly chooses a future feature that requires transmission.

No unnecessary telemetry.

No hidden tracking.

No behavioral profiling for advertising.

=====
FREE-FIRST GUARDIAN
=====

The current project is a personal/free-first project.

The core experience should avoid mandatory paid infrastructure.

However:

FREE DOES NOT MEAN UNLIMITED.

External APIs may have:

quotas
limits
availability restrictions
provider controls.

The application must design around those limitations rather than pretending they do not exist.

=====
TRUTHFULNESS GUARDIAN
=====

NEVER FABRICATE:

videos
thumbnails
statistics
views
channels
subscriptions
history
AI confidence
processing percentages
audio tracks
subtitle tracks
model results
download availability
provider capabilities
performance metrics.

If data is unavailable:

say unavailable.

If capability is unsupported:

say unsupported.

If processing fails:

fallback or explain.

=====

UI GUARDIAN

=====

Every visible element must have a purpose.

Every button must:

work,
or
be intentionally disabled,
or
be clearly experimental.

No decorative dead controls.

No fake premium controls.

No fake download controls.

No fake AI controls.

No excessive badges.

No UI overload.

=====
VISUAL IDENTITY GUARDIAN
=====

U-TUBE:

modern
light
clean
video-centric
white/red visual language.

CINEMORPH:

cinematic
theater
paper
film
warm
vintage-inspired
immersive.

They MUST feel like two different experiences.

Do not merge their visual identities merely for consistency.

Shared infrastructure is acceptable.

Shared visual personality is not required.

=====
RESPONSIVE GUARDIAN

=====

U-Tube:
desktop/tablet/mobile capable.

CineMorph:
primarily larger screens.

CineMorph mobile theater experience is intentionally not a target.

If unsupported:

explain gracefully.

Do not force a broken mobile theater.

=====

FALLBACK GUARDIAN

=====

Every advanced feature must have a fallback.

SMART FRAMING → SAFE/ORIGINAL.

AUDIO ENHANCEMENT → ORIGINAL.

MODEL FAILURE → RULES/STATIC.

THEATER EFFECT FAILURE → BASIC CINEMA VIEW.

REMOTE FAILURE → CACHE WHERE LEGITIMATE.

LOCAL AI FAILURE → BASIC PLAYBACK.

UNSUPPORTED CODEC → CLEAR ERROR.

=====

ARCHITECTURAL GUARDIAN

=====

Do not overengineer the personal version.

Avoid premature:

microservices
distributed systems
Kubernetes
complex cloud infrastructure
unnecessary databases
unnecessary message queues.

Prefer:

modular architecture
clear boundaries
shared services
adapters
workers
local storage
progressive enhancement.

=====
CHANGE GUARDIAN
=====

Before changing existing code:

INSPECT.

Before deleting code:

UNDERSTAND.

Before replacing architecture:

COMPARE.

Before adding a dependency:

JUSTIFY.

Before adding a model:

BENCHMARK.

Before claiming completion:

TEST.

=====

REGRESSION GUARDIAN

=====

Any change to shared infrastructure must be tested against BOTH:

U-TUBE
CINEMORPH.

Any change to playback must verify:

play
pause
seek
resume
navigation
mode changes.

Any change to storage must verify:

history
subscriptions
collections
tickets.

=====

QUALITY GUARDIAN

=====

Inspect the application like a vulture.

Search for:

1px alignment problems
wrong spacing
clipped text
broken icons
layout shifts
overflow
z-index problems
stale state
dead buttons
fake data

console errors
memory leaks
duplicate requests
model leaks
worker leaks
audio drift
video jitter
framing jumps
broken transitions
poor loading states
bad empty states
incorrect errors.

=====

DECISION GUARDIAN

=====

When a decision is not explicitly specified:

choose the solution that best satisfies:

USER VALUE
+
RELIABILITY
+
PERFORMANCE
+
SECURITY
+
PRIVACY
+
MAINTAINABILITY
+
FREE-FIRST.

Do not choose based on:

novelty
complexity
marketing appearance
"industry-like" architecture for its own sake.

=====

SCOPE GUARDIAN

=====

The personal version must remain manageable.

Future ideas may be documented as:

FUTURE
EXPERIMENTAL
RESEARCH.

Do not allow future scope to destabilize the current release.

=====

FINAL GUARDIAN RULE

=====

WHENEVER A FUTURE CHANGE CONFLICTS WITH THE ORIGINAL MOTIVE:

STOP.

RE-EVALUATE.

PRESERVE THE ORIGINAL USER VALUE.

The project should become better,not merely larger.

WHENEVER A FEATURE MAKES THE PRODUCT MORE COMPLEX WITHOUT
MEANINGFULLY IMPROVING THE EXPERIENCE:

DO NOT ADD IT.

WHENEVER A MODEL MAKES PLAYBACK WORSE:

REDUCE OR REMOVE THE MODEL.

WHENEVER A VISUAL EFFECT DISTRACTS FROM THE MOVIE:

REMOVE OR REDUCE THE EFFECT.

WHENEVER A CLAIM CANNOT BE VERIFIED:

DO NOT MAKE THE CLAIM.

WHENEVER A FAILURE CAN BE SAFELY FALLBACKED:

FALLBACK.

WHENEVER A CHANGE RISKS DATA LOSS:

STOP AND PROTECT USER DATA.

FINAL PRINCIPLE:

OMNISTREAM IS NOT MEASURED BY HOW MUCH CODE IT CONTAINS.

OMNISTREAM IS MEASURED BY HOW NATURAL,RELIABLE,PRIVATE,FAST,SAFE,AND
ENJOYABLE IT IS TO USE.

[END_DOCUMENT]

[OMNISTREAM_DOCUMENT_02]
[NAME=U-TUBE_DISCOVERY_PLAYBACK_GUARDIAN]
[TYPE=PRODUCT+TECHNICAL+UX_GUARDIAN]
[PRIORITY=HIGH]
[SCOPE=U-TUBE]
[STATUS=FOUNDATIONAL]

PURPOSE:

This document permanently defines the intended behavior,boundaries,architecture,UX,and
reliability principles of the U-Tube portion of OMNISTREAM.

U-Tube is a personal video discovery and playback environment.

Its purpose is NOT to recreate every capability of a commercial video platform.

Its purpose is to make the user's preferred video-discovery workflow simpler:

SEARCH
→FIND
→WATCH
→SUBSCRIBE
→RETURN
→CONTINUE.

=====
01.CORE PURPOSE
=====

The original problem:

Traditional video-platform usage can contain interruptions, complex recommendation systems, large amounts of unrelated content, and unnecessary navigation.

U-Tube aims to provide a simpler personal viewing environment where the user can:

search for supported online videos,
open supported videos inside the application,
subscribe to channels locally,
build a personal feed,
maintain watch history,
continue unfinished videos,
save collections,
and discover related content.

The system should prioritize user-controlled discovery over engagement optimization.

=====

02.PRODUCT BOUNDARY

=====

U-Tube IS:

a personal video interface,
a discovery layer,
a local subscription manager,
a local history system,
a local collection system,
a playback interface,
a lightweight recommendation system.

U-Tube IS NOT:

a replacement for YouTube infrastructure,
a copy of YouTube's proprietary recommendation engine,
a provider-security bypass,
a DRM circumvention system,
a guarantee that every external video is playable,
a guarantee that provider-controlled advertising or restrictions can be removed,
a fake video database.

=====

03.CORE UX

=====

The ideal experience:

OPEN
↓
SEARCH
↓
RESULTS
↓
SELECT
↓
PLAY
↓
NAVIGATE WITHOUT LOSING PLAYBACK
↓
RETURN
↓
RESUME.

The interface should require as few unnecessary steps as possible.

=====

04.HOME PAGE

=====

Home should represent the user's personal video environment.

Primary sections:

CONTINUE WATCHING
SUBSCRIPTIONS
RECENTLY DISCOVERED
RELATED TO RECENT SEARCHES.

Subscription content should normally receive priority over algorithmic discovery.

=====

05.HOME PRIORITY

=====

Priority order:

1.SUBSCRIBED CHANNEL CONTENT

- 2.CONTINUE WATCHING
- 3.RECENT SEARCH RELATED CONTENT
- 4.OTHER DISCOVERY.

Do not allow recommendations to bury content from subscribed channels.

=====

06.SEARCH

=====

Search is the primary discovery mechanism.

Flow:

```
USER ENTERS QUERY
↓
VALIDATE INPUT
↓
NORMALIZE QUERY
↓
CHECK LOCAL CACHE
↓
REQUEST PROVIDER SEARCH WHEN REQUIRED
↓
VALIDATE RESULTS
↓
NORMALIZE RESULTS
↓
DISPLAY.
```

=====

07.SEARCH INPUT

=====

Search input should support:

normal search query,
supported video URL,
supported direct-link input.

The user should not need to understand the underlying provider architecture.

=====

08.SEARCH UX

=====

While typing:

show recent searches,
local suggestions,
clear action.

Do not perform expensive provider searches on every keystroke.

Search should normally execute intentionally.

=====

09.SEARCH DEBOUNCE

=====

If live search is used:

debounce requests.

Never generate:

REQUEST A
REQUEST A
REQUEST A
REQUEST A

from one user interaction.

=====

10.SEARCH CACHE

=====

Store useful search information locally where appropriate:

query,
results,
timestamp,
provider,
video IDs.

Cached data is an optimization,not permanent truth.

=====

11.STALE SEARCH DATA

=====

If cached results are old:

display only when safe,
mark/handle as cached internally,
refresh when appropriate.

Never pretend stale data is live.

=====

12.SEARCH FAILURE

=====

If provider search fails:

use genuinely cached results if available.

Otherwise:

show a clear empty/error state.

NEVER replace failure with fabricated results.

=====

13.RESULT VALIDATION

=====

Every returned result should be validated before display.

Validate:

video identifier,
title,
channel information,
thumbnail,
provider,
playability information where available.

=====

14.VIDEO IDENTITY

A video should have a stable internal identity:

provider
+
providerVideoid.

Do not use:

title
+
channel

as the primary identity.

Titles can change.

=====

15.VIDEO METADATA

=====

Where available:

title,
channel,
channel identifier,
thumbnail,
duration,
publication date,
description,
views,
provider.

If metadata is unavailable:

do not invent it.

=====

16.THUMBNAILS

=====

Use actual provider thumbnails where legitimately available.

If unavailable:

show neutral placeholder.

NEVER create fake thumbnails simply to fill a grid.

=====

17.VIDEO CARDS

=====

Primary information:

thumbnail
title
channel
duration.

Secondary:

views
published time
progress.

Avoid information overload.

=====

18.VIDEO CARD ACTIONS

=====

Primary:

WATCH.

Secondary:

SAVE
ADD TO COLLECTION.

Optional:

more actions.

Do not create meaningless icons.

=====

19.WATCH PAGE

=====

Hierarchy:

PLAYER

↓

TITLE

↓

CHANNEL

↓

ACTIONS

↓

DESCRIPTION

↓

RELATED.

The player is the dominant element.

=====

20. EMBEDDED PLAYBACK

=====

Where supported, the video should play inside U-Tube rather than forcing unnecessary navigation away from the application.

Use legitimate provider-supported playback mechanisms.

Do not bypass:

provider restrictions,
DRM,
embedding controls,
authentication,
access controls.

=====

21. PROVIDER LIMITATION

=====

External provider behavior remains outside U-Tube's control.

The application must gracefully handle:

embedding unavailable,

video unavailable,
region restriction,
removed video,
provider error,
network failure,
provider policy change.

=====

22.PLAYBACK STATES

=====

Use explicit states:

IDLE
LOADING
READY
PLAYING
PAUSED
BUFFERING
ERROR
ENDED.

Do not allow contradictory UI states.

=====

23.PLAYBACK CONTINUITY

=====

Playback state should survive normal navigation.

Example:

WATCH
→HOME
→SEARCH
→SUBSCRIPTIONS
→HISTORY.

The active playback session should remain intact where technically possible.

=====

24.MINI PLAYER

=====

When leaving the watch page:

FULL PLAYER
→MINI PLAYER.

Playback continues.

Mini player should provide:

play/pause,
expand,
close,
basic playback controls where supported.

=====
25.MINI PLAYER PRIORITY
=====

Mini-player must not obscure:

primary controls,
important search content,
dialogs,
navigation.

Use responsive positioning.

=====
26.MINI PLAYER STATE
=====

The mini player and full player should represent the same playback session.

Avoid creating duplicate media sessions unnecessarily.

=====
27.SEEKING
=====

Seek should update:

player position,
history checkpoint,
UI progress.

Rapid seeking must not trigger unnecessary expensive operations.

=====

28.WATCH POSITION

=====

Persist position periodically and at meaningful events:

pause,
navigation,
page visibility change,
before unload where possible,
periodic checkpoint.

Do not write to storage every animation frame.

=====

29.RESUME

=====

When reopening an unfinished video:

offer:

CONTINUE FROM XX:XX.

If nearly complete:

treat appropriately as almost finished/completed based on defined threshold.

=====

30.HISTORY

=====

History represents actual viewing activity.

Record:

video identity,
last position,
duration where available,
last watched timestamp,
completion state.

=====

31.HISTORY DUPLICATES

=====

Avoid creating duplicate entries merely because the user:

paused,
navigated,
returned,
opened mini player.

Use stable media identity.

=====

32.HISTORY PAGE

=====

Primary:

recently watched.

Secondary:

continue watching.

Actions:

resume,
remove,
clear.

=====

33.CLEAR HISTORY

=====

Clear history should remove stored history.

It must NOT:

delete source media,
delete subscriptions,
delete collections,
delete settings

unless the user explicitly chooses a global reset.

=====

34.SUBSCRIPTIONS

=====

Subscriptions are local user preferences.

The user explicitly adds channels.

U-Tube stores:

channel identity,
channel name,
avatar where available,
provider,
subscription timestamp,
last refresh.

=====

35.SUBSCRIBE FLOW

=====

INPUT CHANNEL

↓

VALIDATE

↓

RESOLVE CHANNEL

↓

SHOW CONFIRMATION

↓

STORE SUBSCRIPTION

↓

REFRESH FEED.

Do not store unresolved channels as valid subscriptions.

=====

36.UNSUBSCRIBE

=====

Unsubscribe should:

remove local subscription.

It should NOT delete:

history,
collections,
previously watched videos.

=====

37.SUBSCRIPTION FEED

=====

Feed:

SUBSCRIPTIONS

↓

LATEST AVAILABLE CONTENT

↓

SORT/PRIORITIZE

↓

DISPLAY.

Do not fabricate missing content.

=====

38.SUBSCRIPTION REFRESH

=====

Refresh should be:

background,
bounded,
cached,
rate-limited.

Do not refresh every time the user opens Home unless necessary.

=====

39.REFRESH CADENCE

=====

The initial design may use approximately:

4-hour refresh intervals.

Implementation may intelligently adjust this based on:

last refresh,
provider limitations,
network availability,
application activity.

The exact interval must not create unnecessary provider load.

=====
40.MANUAL REFRESH
=====

User may manually refresh.

Show:

refreshing.

Do not clear existing content while refreshing.

=====
41.REFRESH FAILURE
=====

Retain previous valid feed.

Show subtle state:

"Could not refresh right now."

Do not erase the user's feed because the network failed.

=====
42.NEW CONTENT
=====

When new subscription videos are available:

do not unexpectedly reorder the entire page while the user is reading.

Show:

NEW VIDEOS AVAILABLE.

Allow user to refresh.

=====

43.RECOMMENDATION SYSTEM

=====

U-Tube intentionally uses a lightweight recommendation system.

It does NOT attempt to reproduce YouTube's proprietary recommendation architecture.

Inputs:

recent searches,
video titles,
subscription content,
watch history where appropriate,
keyword extraction.

=====

44.RECOMMENDATION FLOW

=====

SEARCH/HISTORY/SUBSCRIPTIONS

↓
KEYWORD EXTRACTION
↓
NORMALIZATION
↓
CONTENT SEARCH
↓
FILTER
↓
RANK
↓
CACHE
↓
DISPLAY.

=====

45.KEYWORD EXTRACTION

=====

Extract meaningful terms.

Remove:

punctuation,
common noise,
duplicate terms.

Preserve:

artist,
topic,
technology,
genre,
subject,
specific entities.

=====
46.RECOMMENDATION LIMIT
=====

Recommendation system should remain lightweight.

Do not build a giant machine-learning recommendation infrastructure for the personal version.

=====
47.RECOMMENDATION QUALITY
=====

A recommendation should have a reason.

Possible internal reasons:

related to recent search,
related to watched topic,
from subscribed channel.

Do not invent recommendation reasons.

=====
48.RECOMMENDATION FAILURE
=====

Insufficient data:

show subscription content,
recent content,
or useful discovery content.

Do not fill empty space with fake recommendations.

=====

49.CATEGORIES

=====

Topic chips may exist.

Examples:

Music
Gaming
Coding
News
Technology
Movies
Sports
Education.

But categories must remain useful.

Do not create hundreds of meaningless filters.

=====

50.COLLECTIONS

=====

Provide:

Watch Later
Favorites
Custom Collections.

Collections are local.

=====

51.SAVE FLOW

=====

VIDEO

→SAVE

→CHOOSE COLLECTION

→STORE.

Saving should not interrupt playback.

=====

52.WATCH LATER

=====

Watch Later is simply a collection with special UX.

Do not duplicate storage unnecessarily.

=====

53.FAVORITES

=====

Favorites should represent explicit user action.

Do not automatically favorite watched videos.

=====

54.SETTINGS

=====

Relevant settings:

theme,
autoplay,
playback behavior,
history,
storage,
privacy,
performance,
advanced options.

=====

55.ZERO-CONFIG PRINCIPLE

=====

The normal user should NOT have to:

enter API keys,
configure databases,
install servers,
configure models,
edit code.

Developer configuration may exist separately.

```
=====
56.API CONFIGURATION
=====
```

If an external API is required by the chosen implementation:

keep configuration outside normal consumer UI.

Never expose private credentials in frontend code.

```
=====
57.PROVIDER ADAPTER
=====
```

Provider logic must be isolated.

Conceptual:

```
VideoProvider
├── search()
├── resolve()
├── metadata()
├── capabilities()
└── playback()
```

The UI consumes normalized data.

```
=====
58.PROVIDER NORMALIZATION
=====
```

Convert provider-specific responses into internal objects.

Example:

Video

```
{  
  provider,  
  providerId,  
  title,  
  channel,  
  thumbnail,  
  duration,  
  publishedAt,  
  capabilities  
}
```

Do not allow provider-specific fields to spread throughout the application unnecessarily.

```
=====
```

59.REQUEST DEDUPLICATION

```
=====
```

Identical active requests should be reused.

```
=====
```

60.ABORT STALE REQUESTS

```
=====
```

If user searches:

A
then immediately:

B

results from A must not overwrite B.

```
=====
```

61.RATE LIMITING

```
=====
```

Use:

debounce,
cache,
request deduplication,
backoff,
bounded concurrency.

=====

62.RETRY

=====

Retry only transient failures.

Do not repeatedly retry:

invalid video
invalid URL
permanently unavailable content.

=====

63.EXTERNAL NETWORK

=====

Network requests must not block the entire UI.

Use asynchronous operations.

=====

64.CACHE-FIRST

=====

If valid cached metadata exists:

render it quickly.

Then refresh in background where appropriate.

=====

65.LOCAL STORAGE

=====

Use appropriate storage:

preferences:
small key-value storage.

history/subscriptions/collections:
structured local storage such as IndexedDB where appropriate.

Never store large videos in localStorage.

=====

66.SCHEMA VERSIONING

=====

Persisted U-Tube data requires:

schemaVersion.

Migration must be safe.

=====

67.DATA MIGRATION

=====

Old data:

detect

→migrate

→validate

→save.

If migration fails:

protect existing data.

=====

68.PRIVACY

=====

Default local state:

history

subscriptions

collections

preferences.

Do not transmit user history unnecessarily.

=====

69.SECURITY

=====

Treat all:

URLs
metadata
provider responses
thumbnails
descriptions

as untrusted external data.

70.URL VALIDATION

Never directly trust arbitrary URL input.

Validate:

protocol,
hostname,
provider,
video/channel identifier.

Only construct known-safe provider URLs.

71.IFRAME SECURITY

Only trusted provider domains.

No arbitrary iframe injection.

No:

userURL
→iframe.src

without validation.

72.XSS

Never render provider metadata as raw HTML.

Escape/sanitize.

=====
73.SECRETS
=====

Never store:

API keys,
private tokens,
service credentials

inside client-side source or local storage.

=====
74.LOGGING
=====

Do not log:

secrets,
tokens,
unnecessary personal information,
raw provider credentials.

=====
75.DEPENDENCIES
=====

Use only dependencies with a justified purpose.

Remove:

unused,
duplicate,
unmaintained,
unnecessary packages.

=====
76.PERFORMANCE
=====

U-Tube should prioritize:

fast shell,
fast search response,
fast player startup,
smooth scrolling,
low memory.

=====
77.LAZY LOADING
=====

Do not load:

CineMorph models,
theater assets,
advanced analysis engines

when only U-Tube is being used.

=====
78.CODE SPLITTING
=====

U-Tube and CineMorph should be separated where practical.

Opening U-Tube should not require loading the entire CineMorph engine.

=====
79.THUMBNAIL PERFORMANCE
=====

Lazy-load below-the-fold thumbnails.

Reserve dimensions to avoid layout shifts.

=====
80.FEED PERFORMANCE
=====

Do not render thousands of cards simultaneously.

Use pagination/virtualization where justified.

=====
81.SCROLL PERFORMANCE

=====

Avoid expensive per-card computations during scroll.

=====

82.UI STATE

=====

Do not place every transient UI state in global state.

Local:

hover,
menus,
temporary dialogs.

Global:

active playback,
persistent user state.

=====

83.PLAYBACK STATE

=====

Central playback controller should own:

current media,
position,
duration,
state,
volume,
audio,
subtitle,
engine.

=====

84.PLAYBACK EVENTS

=====

Handle:

play,
pause,

timeupdate,
buffering,
ended,
error,
fullscreen.

Do not update persistent storage on every timeupdate.

Throttle checkpoints.

=====
85.ERROR UX
=====

Errors must explain:

what happened,
what user can do.

Example:

"This video cannot be played inside U-Tube."

Possible action:

"Open supported playback"
or
"Try another video."

Never expose technical stack traces.

=====
86.EMPTY STATES
=====

Search:

"No matching videos found."

Subscriptions:

"Subscribe to channels to build your feed."

History:

"Videos you watch will appear here."

Collections:

"Save videos to organize them."

=====
87.LOADING STATES
=====

Use actual loading indicators.

Do not fabricate progress.

Skeletons are allowed for content loading.

=====
88.NO FAKE DATA
=====

Absolutely prohibited:

fake video results,
fake thumbnails,
fake channels,
fake views,
fake history,
fake subscription data,
fake recommendation data.

Development fixtures must be isolated.

=====
89.NO FAKE AI
=====

Do not label deterministic sorting as AI.

Do not show AI badges without actual AI.

=====
90.NO FAKE ANALYTICS
=====

Statistics must come from actual stored playback data.

=====

91.NO FAKE DOWNLOAD

=====

Do not expose download functionality unless a legitimate, technically supported implementation exists.

Do not imply that arbitrary provider videos can be downloaded.

=====

92.NO ADVERTISEMENT CLAIM

=====

U-Tube should not introduce its own advertisements.

However, the application must not claim control over advertisements that are generated by an external provider when using that provider's supported playback system.

Do not build unsupported mechanisms intended to circumvent provider advertising or access controls.

=====

93.ACCESSIBILITY

=====

Search:

keyboard accessible.

Cards:

keyboard accessible.

Buttons:

accessible labels.

Menus:

focus management.

Dialogs:

focus trap/return.

=====
94.KEYBOARD
=====

Support:

Enter

Space

Escape

Arrow keys where appropriate.

Do not override browser shortcuts unnecessarily.

=====
95.RESPONSIVE
=====

Desktop:

full navigation.

Tablet:

compressed.

Mobile:

single/limited columns.

Mini player:

responsive.

=====
96.MOBILE
=====

U-Tube remains usable on mobile.

CineMorph is separate and should not be forced into mobile theater mode.

=====

97.THEME

=====

U-Tube primary visual identity:

light
white
red
clean
modern.

Do not import CineMorph textures or cinematic styling.

=====

98.NAVIGATION

=====

Primary:

Home
Search
Subscriptions
History
Collections.

Optional:

Settings.

=====

99.ENGINE SWITCH

=====

Allow transition:

U-Tube
↔
CineMorph.

Do not unnecessarily destroy active playback state.

=====

100.ROUTE SAFETY

=====

Direct routes should handle:

invalid video ID,
missing video,
missing history item,
missing collection,
deleted subscription.

Never crash.

=====

101.BROWSER BACK

=====

Respect normal browser navigation.

Do not trap the user.

=====

102.SESSION RESTORATION

=====

On application reopen:

restore useful local state.

Do not automatically begin playback unless the user has explicitly enabled autoplay and browser rules allow it.

=====

103.AUTOPLAY

=====

Autoplay behavior must respect:

browser restrictions,
user preference,
provider restrictions.

Never assume autoplay is guaranteed.

=====

104.VOLUME

=====

Persist user preference locally where useful.

Do not unexpectedly reset volume.

=====

105.AUDIO

=====

U-Tube should use provider-supported audio behavior.

Do not promise:

Dolby
Atmos
lossless
spatial

unless actually supported by the playback path and device.

=====

106.FULLSCREEN

=====

Use browser-supported fullscreen APIs where available.

Handle permission/user-gesture limitations gracefully.

=====

107.PICTURE-IN-PICTURE

=====

Optional only if technically supported.

Never display a dead PiP button.

=====

108.MEDIA SESSION

=====

Where browser support allows:

integrate with Media Session APIs.

Do not require it.

=====

109.PERFORMANCE DEGRADATION

=====

If device is weak:

reduce UI effects.

Never reduce basic usability first.

=====

110.NETWORK DEGRADATION

=====

Fast:

normal.

Slow:

cache-first.

Offline:

local history/subscriptions remain accessible.

Remote discovery:

requires network unless cached.

=====

111.FEED OFFLINE

=====

Previously cached content may be displayed.

Label/handle stale state internally.

Do not fabricate new videos.

=====

112.HISTORY OFFLINE

=====

History must remain available offline.

=====

113.SUBSCRIPTIONS OFFLINE

=====

Subscription list remains available.

Latest remote content requires network.

=====

114.COLLECTIONS OFFLINE

=====

Collections remain available.

=====

115.WATCH POSITION OFFLINE

=====

Local state may continue updating.

=====

116.ERROR ISOLATION

=====

Recommendation failure must NOT stop playback.

Subscription refresh failure must NOT erase history.

History failure must NOT stop video playback.

=====

117.ASYNC SAFETY

=====

All async tasks must verify they are still relevant before committing results.

=====

118.CONCURRENCY

Bound:

search requests
refreshes
metadata calls.

119.LONG SESSION

U-Tube must remain stable during:

hours of browsing,
long playback,
repeated navigation.

120.MEMORY

Release:

object URLs
event listeners
media resources
temporary objects.

121.VISUAL QA

Inspect:

alignment,
spacing,
text,
icons,
thumbnail aspect,
duration badges,
progress bars,
mini player,

sidebar,
search,
empty states,
errors.

=====

122.VULTURE QA

=====

Look specifically for:

1px misalignment,
incorrect icon size,
text truncation,
broken hover,
incorrect focus,
layout shifts,
overlapping mini-player,
stale loading state,
duplicate cards,
incorrect timestamps,
wrong progress,
scroll issues.

=====

123.FUNCTIONAL QA

=====

Test:

search
watch
pause
resume
seek
navigation
mini-player
subscribe
unsubscribe
refresh
history
collections
settings.

=====

124.FAILURE QA

=====

Test:

invalid URL
invalid video
removed video
network failure
provider failure
slow network
offline
storage failure
browser refresh
rapid navigation.

=====

125.STRESS QA

=====

Test:

many searches
many history entries
many subscriptions
many collections
long sessions
rapid navigation.

=====

126.NO DATA LOSS

=====

A failed refresh must not erase valid local data.

A failed migration must not destroy old data.

A failed provider call must not corrupt playback state.

=====

127.CHANGE MANAGEMENT

=====

Before modifying working U-Tube code:

inspect dependencies.

After modification:

run regression.

```
=====
128.NO BLIND REWRITE
=====
```

Do not rebuild U-Tube merely to make its source code look cleaner.

Measure actual problems first.

```
=====
129.FEATURE PRIORITY
=====
```

P0:

playback
search
data integrity
security.

P1:

subscriptions
history
collections
navigation.

P2:

recommendations
performance
polish.

P3:

experimental features.

=====

130.RECOMMENDATION MODEL LIMIT

=====

Do not allow recommendation engineering to consume more resources than playback.

=====

131.SEARCH MODEL LIMIT

=====

Use deterministic/provider search first.

AI query understanding is optional.

=====

132.LLM LIMIT

=====

An LLM may help interpret complex queries,but it must not replace deterministic search.

Example:

USER:
"latest beginner Java tutorials"

LLM may extract:

topic=Java
level=beginner
intent=tutorial
recency=latest.

Provider search performs actual retrieval.

=====

133.HYBRID SEARCH

=====

Potential:

query parser
+
provider search
+

local ranking.

Do not use an LLM to invent results.

=====

134.RANKING

=====

Rank using actual available signals:

relevance
subscription status
recency
local history
user interaction.

Do not fabricate unavailable metrics.

=====

135.USER CONTROL

=====

The user should be able to:

clear history,
unsubscribe,
remove recommendations where implemented,
clear cache,
reset U-Tube state.

=====

136.RESET

=====

Reset U-Tube:

history
subscriptions
collections
preferences

only according to explicit user choice.

Never delete unrelated CineMorph data unless global reset is selected.

=====

137.SHARED STATE

=====

Shared:

active playback
media identity
global settings where appropriate.

Separate:

U-Tube history
CineMorph tickets.

=====

138.FUTURE EXTENSION

=====

Possible future:

more providers,
better query understanding,
advanced recommendation,
cloud synchronization,
accounts,
shared libraries.

These remain future.

Do not destabilize the personal version to implement them early.

=====

139.FREE-FIRST EXTENSION

=====

Future infrastructure must preserve the ability to operate locally/free-first.

=====

140.PROVIDER CHANGE

=====

If provider behavior changes:

adapt provider adapter.

Do not spread provider-specific changes throughout the UI.

=====

141.PROVIDER REMOVAL

=====

If a provider becomes unavailable:

U-Tube should fail gracefully.

The application should not crash.

=====

142.DOCUMENTATION

=====

Document:

search flow
provider adapter
storage
history
subscriptions
recommendations
playback
limitations.

=====

143.NO MISLEADING DOCUMENTATION

=====

Documentation must not claim:

universal playback
universal downloads
unlimited provider access
guaranteed ad removal
unsupported AI capabilities.

=====

144.RELEASE CHECK

=====

Before declaring U-Tube complete:

- ☐ HOME WORKS
- ☐ SEARCH WORKS
- ☐ REAL RESULTS WORK
- ☐ PLAYBACK WORKS WHERE SUPPORTED
- ☐ MINI PLAYER WORKS
- ☐ HISTORY WORKS
- ☐ RESUME WORKS
- ☐ SUBSCRIPTIONS WORK
- ☐ FEED REFRESH WORKS
- ☐ COLLECTIONS WORK
- ☐ SETTINGS WORK
- ☐ ERROR STATES WORK
- ☐ EMPTY STATES WORK
- ☐ OFFLINE LOCAL STATE WORKS
- ☐ NO FAKE DATA
- ☐ NO DEAD CORE BUTTONS
- ☐ NO CRITICAL CONSOLE ERRORS
- ☐ SECURITY REVIEW PASSED
- ☐ PERFORMANCE REVIEW PASSED
- ☐ VISUAL REVIEW PASSED.

=====

145.FINAL GUARDIAN RULE

=====

U-Tube must always remain:

SIMPLE TO SEARCH.
EASY TO WATCH.
EASY TO RETURN TO.
EASY TO CONTROL.
HONEST ABOUT LIMITATIONS.
PRIVATE BY DEFAULT.
FREE-FIRST.
FAST.
RELIABLE.

If a future feature makes U-Tube:

slower,

more confusing,
more dependent on paid infrastructure,
less private,
less reliable,
more cluttered,
or less truthful,

the feature must be rejected,redesigned,or made optional.

FINAL PRINCIPLE:

U-Tube is successful when the user stops thinking about the technology and simply thinks:

"I searched for something,and I watched it."

[END_OMNISTREAM_DOCUMENT_02]

[OMNISTREAM_DOCUMENT_03]

[NAME=CINEMORPH_THEATER_INTELLIGENT_MEDIA_GUARDIAN]

[TYPE=PRODUCT+MEDIA+AI+UX+PERFORMANCE_GUARDIAN]

[PRIORITY=HIGH]

[SCOPE=CINEMORPH]

[STATUS=FOUNDATIONAL]

PURPOSE:

This document is the permanent guardian of the CineMorph vision,cinematic presentation system,intelligent framing architecture,local-media workflow,theater simulation,AI behavior,audio behavior,performance constraints,and failure philosophy.

CineMorph exists to transform supported media playback into a theatrical viewing experience on larger screens.

The goal is NOT to pretend that a laptop,TV,or monitor physically becomes an IMAX theater.

The goal is to use software,geometry,media analysis,rendering,animation,audio processing,and presentation design to create the strongest technically honest theatrical experience available on the user's hardware.

=====

01.CORE EXPERIENCE

=====

CINEMORPH EXPERIENCE:

SELECT SOURCE
↓
PREPARE MEDIA
↓
GENERATE TICKET
↓
PREPARE CINEMA
↓
CURTAINS
↓
INTRO
↓
SCREEN OPENS
↓
MOVIE
↓
CINEMATIC PRESENTATION
↓
ENDING
↓
CURTAINS
↓
SESSION/TICKET SAVED.

The user should feel that they entered a theater rather than merely opened another video player.

However:

THE MOVIE REMAINS THE PRIMARY CONTENT.

=====

02.SOURCE TYPES

=====

CineMorph supports two conceptual sources:

- 1.LOCAL FILE
- 2.SUPPORTED ONLINE VIDEO LINK.

LOCAL MEDIA IS THE PRIMARY FOCUS.

Online playback is secondary and must respect provider capabilities.

=====

03.LOCAL-FIRST PRINCIPLE

=====

Local files should receive the strongest CineMorph feature set because local playback allows greater control over:

analysis
timing
framing
audio
subtitle handling
rendering
offline operation.

=====

04.LOCAL FILE PRIVACY

=====

Local media should remain on the user's device whenever technically possible.

Do not upload a movie merely to perform an operation that can be performed locally.

=====

05.MEDIA SELECTION

=====

CineMorph landing page:

SELECT LOCAL FILE
OR
PASTE ONLINE VIDEO LINK.

The choice must be obvious.

Do not make the user understand technical media pipelines.

=====

06.MEDIA METADATA

=====

Before playback,extract where technically possible:

filename

duration
resolution
width
height
aspect ratio
frame rate
video codec
audio tracks
subtitle tracks
chapters.

Missing information:

UNKNOWN.

Never fabricate metadata.

=====
07.MEDIA PREPARATION
=====

Preparation should happen asynchronously.

Possible pipeline:

SELECT
→METADATA
→CAPABILITY CHECK
→MODEL PREPARATION
→SAFE PLAYBACK
→OPTIONAL INTELLIGENCE.

Never force the user to wait for the entire movie to be analyzed before playback.

=====
08.THREE PRIMARY PRESENTATION MODES
=====

CineMorph intentionally limits the core presentation choices to:

IMAX
1.90:1

TRUE IMAX

1.43:1

ORIGINAL

source-native presentation.

=====

09.IMAX MODE

=====

Default theater mode:

1.90:1.

It represents the wide large-format presentation.

=====

10.TRUE IMAX MODE

=====

1.43:1.

The system attempts to produce a tall large-format viewing aperture using intelligent presentation where possible.

It must not claim to recover image content that does not exist in the source.

=====

11.ORIGINAL MODE

=====

Original preserves the source presentation as closely as technically possible.

It is the ultimate fallback.

=====

12.ORIGINAL 4:3 OPTION

=====

Where supported by the product design:

Original 4:3 presentation may be offered as a simple crop/presentation mode.

This must remain distinct from intelligent cinematic reframing.

=====

13.MOBILE POLICY

=====

CineMorph theater mode is NOT a mobile-first experience.

Do not force the theatrical environment into a tiny mobile viewport.

On unsupported mobile configurations:

provide a clear fallback or normal media playback.

=====

14.ADAPTIVE THEATER

=====

Theater geometry must respond to:

viewport size
screen dimensions
aspect ratio
device pixel ratio
fullscreen state.

Never use fixed desktop-only coordinates.

=====

15.THEATER LAYERS

=====

Conceptual rendering layers:

LAYER 1:
background/environment.

LAYER 2:
theater architecture.

LAYER 3:
chairs/seating.

LAYER 4:
curtains/side elements.

LAYER 5:
screen frame/aperture.

LAYER 6:
video presentation plane.

LAYER 7:
lighting/ambient effects.

LAYER 8:
controls/interaction.

The exact implementation may vary.

=====

16.SCREEN APERTURE

=====

The selected aspect ratio defines the visible aperture.

The video rendering system should position the media behind or within this aperture depending on the implementation.

=====

17."SCREEN BEHIND THE HOLE" CONCEPT

=====

Conceptual model:

THEATER FRAME
+
VISIBLE APERTURE
+
VIDEO PLANE BEHIND APERTURE.

The video plane may move smoothly underneath the fixed presentation frame.

This creates the perception that the theater screen is revealing the best part of the source image.

=====

18.NO VISUAL LEAK

=====

The video must never visibly escape the intended aperture.

No:

edges
black artifacts
unintended image regions
layer gaps
mask leaks.

=====

19.FRAME CANDIDATE SYSTEM

=====

The intelligent framing engine should not directly choose one position from one model.

It should generate candidate crops/positions.

Each candidate is evaluated.

=====

20.FRAME ANALYSIS INPUT

=====

Possible signals:

faces
persons
objects
saliency
motion
scene boundaries
composition
text
subtitles
foreground
background
camera movement
visual center.

=====

21.FACE ANALYSIS

=====

Detect:

face location
size
confidence
movement.

Multiple faces should be considered collectively.

=====

22.OBJECT ANALYSIS

=====

Important objects may influence framing.

Examples:

vehicle
weapon
animal
sports object
main prop.

Do not blindly center every detected object.

=====

23.SALIENCY

=====

Saliency provides an additional signal.

It should not become the only framing authority.

=====

24.SUBJECT IMPORTANCE

=====

The engine should infer:

primary
secondary
background.

Where semantic certainty is low:

use conservative framing.

=====

25.FRAME-WITHIN-A-FRAME

=====

The system may use existing visual geometry:

doors
windows
arches
branches
frames.

But it must NOT move the source in an unnatural way merely because geometry exists.

=====

26.LEADING LINES

=====

Lines can influence composition scoring:

roads
tracks
walls
architecture.

The system may prefer candidates that preserve useful leading lines.

=====

27.DIAGONALS

=====

Diagonal composition can indicate:

motion
energy
action.

Do not automatically straighten diagonals.

=====

28.GEOMETRIC SHAPES

=====

Triangles
circles
squares
arches

may influence composition.

These are heuristic signals, not mandatory rules.

=====

29. SCREEN DIRECTION

=====

Subject movement and gaze direction can influence lookroom.

Do not blindly assume:

left-to-right = good
right-to-left = bad.

These are cinematic heuristics, not universal laws.

=====

30. LOOKROOM

=====

When a subject looks/moves toward one direction:

prefer sufficient space in that direction.

Avoid placing the subject against the aperture edge without reason.

=====

31. HEADROOM

=====

Maintain reasonable headroom.

Avoid:

excessive empty space
clipped head

unnatural crop.

=====

32.LEADING SPACE

=====

Moving subjects require sufficient visual space where appropriate.

=====

33.VISUAL DEPTH

=====

Foreground/background information should be preserved when it contributes meaningfully to the scene.

=====

34.VANISHING POINTS

=====

Perspective geometry may influence composition scoring.

Do not overfit to geometric lines.

=====

35.DUTCH ANGLE

=====

Do not automatically correct or reinterpret camera angles.

A Dutch angle may be intentional.

=====

36.CINEMATOGRAPHY PRESERVATION

=====

CineMorph should enhance presentation without rewriting the cinematographer's intent.

When source composition is already strong:

DO LESS.

=====

37.SOURCE RESPECT

=====

The original source is the reference truth.

CineMorph may:

crop
position
scale
present
enhance audio
decorate.

It should not:

invent subjects
invent scenes
invent frames
rewrite story content.

=====

38.SCENE DETECTION

=====

Detect meaningful shot/scene transitions.

At hard cuts:

reset or rapidly adapt framing context.

Do not interpolate framing across unrelated shots.

=====

39.FAST CUTS

=====

If a shot is extremely short:

avoid expensive analysis.

Use conservative framing.

=====

40.STATIC SHOTS

=====

For static scenes:

reduce unnecessary movement.

The frame should remain stable.

=====

41.CAMERA MOVEMENT

=====

Estimate camera motion where possible.

Avoid fighting the source camera movement.

=====

42.SUBJECT TRACKING

=====

Tracking should provide temporal continuity.

Do not treat each frame independently.

=====

43.TEMPORAL SMOOTHING

=====

Position changes must be smoothed.

Avoid:

jitter
micro-corrections
oscillation.

=====

44.HYSTERESIS

=====

New candidate must be sufficiently better before replacing the current candidate.

This prevents unnecessary movement.

=====

45.VELOCITY LIMIT

=====

Limit:

translation speed
scale speed
acceleration.

=====

46.EMERGENCY TRANSITION

=====

Allow faster transition only when necessary.

Examples:

subject leaves frame
important object enters
current crop becomes invalid.

=====

47.FRAME SCORING

=====

Candidate score may combine:

subject coverage
face retention
lookroom
headroom
saliency
composition
text safety
subtitle safety
motion continuity
crop severity
cinematic stability.

=====

48.WEIGHT ADAPTATION

=====

Weights should change by scene.

Dialogue:
face/eye-line priority.

Action:
motion/subject priority.

Landscape:
composition/environment priority.

Text:
text preservation priority.

Credits:
minimal intelligent movement.

=====

49.CONFIDENCE

=====

Every intelligent decision should have confidence.

Low confidence:

fallback to stable framing.

=====

50.FALLBACK FRAMING

=====

Hierarchy:

AI/HYBRID
→RULES
→CURRENT SAFE FRAME
→CENTERED CROP
→ORIGINAL.

=====

51.NO AI BLOCKING

=====

The movie must never wait indefinitely for AI.

Playback starts with safe presentation.

AI improves it asynchronously.

=====

52.LOOKAHEAD

=====

Use bounded lookahead.

Possible architecture:

PLAYBACK T
ANALYZE T+small window.

Never analyze the entire movie in advance merely for framing.

=====

53.PREPARATION PIPELINE

=====

TICKET/INTRO TIME
+
SHORT PREBUFFER
+
AI WARM-UP
+
FIRST SCENE ANALYSIS.

Use this time intelligently.

Do not fake processing.

=====

54.TICKET ANIMATION

=====

Ticket should include:

movie title
runtime
theater name
seat number
date/time where appropriate

presentation mode.

Random seat numbers must be decorative and deterministic enough to avoid unnecessary state changes.

=====

55.TICKET PERSISTENCE

=====

Save:

media identity
source type
position
presentation mode
audio selection
subtitle selection
ticket information.

=====

56.TICKET RESUME

=====

If user leaves mid-movie:

save ticket/session.

Returning user can select:

CONTINUE.

=====

57.MISSING LOCAL FILE

=====

If the file no longer exists:

show:

"Source file unavailable."

Offer:

RESELECT FILE.

Never silently replace it.

=====

58.TICKET TEAR

=====

Ticket tear is presentation only.

It must not block playback indefinitely.

=====

59.CURTAIN OPENING

=====

Suggested:

ticket
→curtain
→screen reveal
→intro
→movie.

Transitions must remain short.

=====

60.INTRO VIDEO

=====

Intro should appear on the theater screen.

Do not show a fake external cinema screen containing another cinema.

The user's screen IS the theater presentation surface.

=====

61.INTRO LENGTH

=====

Default target:

short.

Approximately around 10 seconds where appropriate.

Do not force long intros every time.

Allow future preference:

skip intro.

=====

62.DEFAULT MODE

=====

CineMorph default:

IMAX 1.90:1.

The user may change mode.

=====

63.FULLSCREEN

=====

IMAX/True IMAX theater mode should encourage fullscreen.

If fullscreen fails:

remain in adaptive theater viewport.

Do not crash.

=====

64.THEATER SEATING

=====

Seats may occupy lower presentation area.

They must not cover important movie content.

=====

65.SCREEN CURVATURE

=====

Curved screen presentation may be used for theater aesthetics.

Avoid geometric distortion of the source unless intentionally implemented and clearly beneficial.

=====

66.AMBIENT LIGHT

=====

Optional ambient lighting may derive from video content.

Keep intensity subtle.

Do not distract.

=====

67.AMBIENT COLOR

=====

Use dominant colors carefully.

Avoid rapid flashing.

=====

68.SPEAKERS

=====

Visual speaker props may exist.

They are decorative.

Do not claim physical speaker simulation.

=====

69.AUDIO ENHANCEMENT

=====

Possible processing:

equalization
dynamic range adjustment
spatialization where supported
dialogue enhancement
cinema profile.

=====

70.AUDIO PRIORITY

=====

SYNC > LOUDNESS > EFFECTS.

Never introduce processing that creates noticeable audio delay.

=====

71.ORIGINAL AUDIO FALLBACK

=====

If processing fails:

original audio.

=====

72.MULTI-AUDIO

=====

Local files may contain:

multiple language tracks
commentary tracks
alternate mixes.

Expose available tracks.

Do not fabricate unavailable languages.

=====

73.SUBTITLE TRACKS

=====

Support available subtitle tracks where technically possible.

=====

74.SUBTITLE PROTECTION

=====

Smart framing should avoid clipping subtitles.

Subtitle regions receive high safety priority.

=====

75.LOCAL FORMAT SUPPORT

=====

Support formats the browser/runtime can actually decode.

For unsupported formats:

provide clear explanation and, where technically feasible, an optional local conversion path.

Never claim "every format" without testing.

=====

76.REMUX VS TRANSCODE

=====

Prefer remux when possible.

Transcode only when necessary.

=====

77.LARGE FILE RULE

=====

A 3-hour movie must NOT be fully loaded into RAM.

Use:

native media streaming
bounded buffers
incremental processing.

=====

78.LONG MOVIE ANALYSIS

=====

Do not process:

3 hours × every frame

unless explicitly necessary.

Use:

scene sampling
adaptive analysis
short lookahead

tracking.

=====

79.ANALYSIS RATE

=====

Analysis frequency should adapt to:

motion
scene complexity
device capability.

Fast action:

more frequent.

Static scene:

less frequent.

=====

80.MODEL ROUTER

=====

Model Router chooses:

model
resolution
frequency
backend.

Consider:

CPU
GPU
WebGPU
WASM
memory.

=====

81.WORKERS

=====

Heavy vision processing should run outside the main UI thread where possible.

=====

82.WORKER LIFECYCLE

=====

Start when CineMorph requires it.

Terminate when session ends.

=====

83.MODEL LOADING

=====

Do not load every model immediately.

Use progressive loading.

=====

84.MODEL CACHE

=====

Models must be:

versioned
validated
bounded.

=====

85.MODEL FAILURE

=====

Model failure:

fallback.

Never stop movie playback.

=====

86.HYBRID MODEL ARCHITECTURE

=====

Preferred structure:

DETECTOR

+
TRACKER
+
SALIENCY
+
COMPOSITION ENGINE
+
RULE ENGINE
+
TEMPORAL FILTER
+
RENDERER.

A single model should not be responsible for everything.

=====
87.CUSTOM MODEL RULE
=====

Build a custom model only when:

existing models fail a measurable requirement.

=====
88.LLM ROLE
=====

Potential LLM roles:

scene semantic classification
configuration
offline scene summarization
developer assistant.

Not:

frame-by-frame control.

=====
89.REAL-TIME CONTROL
=====

Real-time control should use:

fast deterministic logic
specialist models
tracking
geometry.

=====

90.AI MODEL LIMIT

=====

No model should be allowed to consume enough resources to make the movie lag.

=====

91.PERFORMANCE PROFILES

=====

AUTO:
balanced.

PERFORMANCE:
reduced analysis.

CINEMATIC:
maximum safe analysis.

SAFE:
minimal AI.

=====

92.AUTO ADAPTATION

=====

If FPS drops:

reduce analysis.

If memory increases:

reduce cache/model footprint.

If GPU overloads:

reduce inference/effects.

=====

93.FRAME RATE

=====

Do not confuse:

video FPS
render FPS
analysis FPS.

They are different.

=====

94.RENDER LOOP

=====

Rendering should remain independent from analysis completion.

The renderer always has a valid frame state.

=====

95.ANALYSIS QUEUE

=====

Use bounded queue.

Discard obsolete analysis requests when the playback position moves forward.

=====

96.SEEK

=====

On seek:

cancel future analysis
reset tracking
analyze new region
resume.

=====

97.MODE SWITCH

=====

Switching:

1.90

↔1.43

↔Original

should preserve:

playback position

audio

subtitle

session.

=====

98.RESIZE

=====

Resize changes:

aperture geometry.

It should NOT trigger unnecessary re-analysis of the movie content.

=====

99.FRAMING ENGINE API

=====

Conceptual:

analyze(frame/window)

→detections.

generateCandidates()

→candidateFrames.

scoreCandidates()

→scores.

select()

→targetFrame.

smooth()

→renderFrame.

=====

100.FRAME REPRESENTATION

=====

Represent target using normalized coordinates where practical.

Avoid hardcoded pixel positions.

=====

101.ASPECT GEOMETRY

=====

The aperture should be mathematically derived from:

viewport
target aspect ratio.

Do not manually tune every screen size.

=====

102.SAFE ZONES

=====

Maintain:

subject safe zone
subtitle safe zone
text safe zone.

=====

103.CROP LIMIT

=====

Avoid excessive crop.

If acceptable framing cannot be achieved:

reduce intelligent framing
or fallback.

=====

104.UPSCALE LIMIT

=====

Do not aggressively enlarge low-resolution video merely to fill the screen.

Quality degradation must be considered.

=====

105.SOURCE QUALITY

=====

Analyze:

resolution
bitrate where available
aspect ratio
compression.

Adapt expectations accordingly.

=====

106.1.43 LIMITATION

=====

If source is wide and lacks vertical content:

do not pretend true 1.43 content exists.

Use:

safe crop
letterboxing
original mode
or conservative presentation.

=====

107.1.90 LIMITATION

=====

Same principle applies.

Do not destroy source composition merely to fill every pixel.

=====

108.ORIGINAL PRIORITY

=====

If intelligent presentation repeatedly produces worse results:

original mode remains the correct answer.

=====

109.CINEMATIC PRINCIPLE

=====

The best frame is not necessarily:

the frame with the face at center.

It is the frame that best preserves:

story-relevant subject

composition

direction

context

continuity.

=====

110.CINEMATOGRAPHY RULE

=====

Rules are heuristics.

They are not absolute laws.

Context overrides simplistic rules.

=====

111.MOVEMENT

=====

Movement should be:

smooth

predictable

minimal

context-aware.

=====

112.NO CAMERA-LIKE JITTER

=====

Do not create artificial camera shake.

=====

113.NO UNNECESSARY ZOOM

=====

Scale changes should occur only when meaningful.

=====

114.NO CONSTANT REFRAMING

=====

If current composition remains good:

hold.

=====

115.NO FRAME HUNTING

=====

Do not continuously search for a marginally better frame.

=====

116.SHOT BOUNDARIES

=====

At scene cuts:

reset target.

=====

117.CINEMATIC CONTINUITY

=====

Avoid abrupt frame changes unless the source itself changes abruptly.

=====

118.TEXT/LOGO SAFETY

=====

Protect:

titles

logos

watermarks

credits
important signs.

=====

119.CREDITS

=====

Credits should generally use:

stable framing.

Do not aggressively crop names.

=====

120.LETTERBOX HANDLING

=====

Detect cinematic bars where practical.

Do not accidentally treat black bars as important content.

=====

121.SOURCE MATTE

=====

Distinguish:

actual image
letterbox
pillarbox.

=====

122.MULTIPLE ASPECT SCENES

=====

A movie may change aspect ratio intentionally.

CineMorph should detect/handle such changes where possible.

=====

123.INTENTIONAL ASPECT CHANGE

=====

Do not fight intentional cinematic transitions.

=====

124.VIDEO OUTPUT

=====

Rendering should preserve:

sharpness
correct aspect
correct color behavior where supported
sync.

=====

125.COLOR

=====

Do not alter cinematic color grading unless an explicit enhancement mode exists.

=====

126.HDR

=====

Handle browser/device HDR capabilities honestly.

Do not fake HDR.

=====

127.AUDIO LATENCY

=====

Any DSP must be measured.

If latency exceeds acceptable synchronization:

fallback.

=====

128.AUDIO TRACK SWITCH

=====

Track changes should:

preserve timestamp
avoid unnecessary reload

maintain sync.

=====

129.SUBTITLE SWITCH

=====

Subtitle changes should:

preserve timestamp
preserve framing.

=====

130.THEATER CONTROL

=====

Controls should remain visually subtle.

They should disappear when inactive where appropriate.

=====

131.CONTROL PRIORITY

=====

Playback:

play/pause
seek
volume
fullscreen
settings.

CineMorph:

mode
audio
subtitle
performance.

Do not overcrowd the control bar.

=====

132.THEATER CLEANLINESS

=====

During movie playback:

minimal UI.

The screen should dominate.

=====
133.CURTAIN EXIT
=====

At movie completion:

fade/curtain sequence.

Do not block user controls unnecessarily.

=====
134.POST-SHOW
=====

Possible:

ticket
title
runtime
resume/rewatch
return.

Avoid excessive menus.

=====
135.SESSION PERSISTENCE
=====

Save session periodically.

Do not wait until browser close.

=====
136.AUTOSAVE
=====

Checkpoint:

every reasonable interval
pause
mode switch
audio switch
navigation.

Avoid per-frame storage.

=====

137.SESSION RESTORE

=====

When reopening:

identify media.

For local files:

require re-selection if browser security prevents persistent file access.

=====

138.FILE HANDLE

=====

Where supported and explicitly authorized:

persistent file handles may improve resume.

But permission must be respected.

=====

139.NO FILE UPLOAD

=====

Do not upload local movies silently.

=====

140.LOCAL SECURITY

=====

Treat local files as untrusted.

Do not execute content.

=====

141.MALFORMED FILE

=====

Malformed media:

fail safely.

Never freeze entire application.

=====

142.HUGE FILE

=====

Huge media:

stream/process incrementally.

Do not copy unnecessarily.

=====

143.STORAGE QUOTA

=====

Temporary generated data must be bounded.

Cleanup after session.

=====

144.OBJECT URL CLEANUP

=====

Revoke temporary object URLs when no longer required.

=====

145.FRAME BUFFER CLEANUP

=====

Release:

VideoFrame
ImageBitmap
canvas resources

when obsolete.

=====

146.WORKER CLEANUP

=====

Terminate workers on session completion.

=====

147.AUDIO NODE CLEANUP

=====

Disconnect unused audio nodes.

=====

148.MEMORY TEST

=====

Test:

30 min

1 hr

2 hr

3 hr.

Check memory trend.

=====

149.REPEATED TEST

=====

Open and close CineMorph repeatedly.

Check:

workers

media

object URLs

audio

GPU resources.

=====

150.SEEK TEST

=====

Rapid seeking must not leave stale analysis.

=====

151.MODE SWITCH TEST

=====

Repeated mode switching must not:

reload unnecessarily
lose position
break audio
break theater.

=====

152.RESIZE TEST

=====

Resize:

small
large
ultrawide
fullscreen.

Verify aperture.

=====

153.MULTI-MONITOR

=====

Moving browser between monitors should preserve theater experience.

=====

154.BROWSER SLEEP

=====

After system sleep:

revalidate timing/rendering.

=====

155.NETWORK INTERRUPTION

=====

Local:

continue.

Remote:

follow provider/browser behavior.

Do not fabricate uninterrupted remote playback.

=====

156.OFFLINE MODE

=====

Offline mode is primarily a local-media capability.

Do not claim offline YouTube playback unless technically and legitimately supported.

=====

157.MODEL OFFLINE

=====

If local models are installed:

CineMorph may continue intelligent processing offline.

=====

158.MODEL DOWNLOAD

=====

Model downloads should:

show real progress
validate integrity
support retry.

=====

159.MODEL STORAGE

=====

Do not repeatedly download the same model.

=====

160.MODEL UPDATE

=====

Verify:

version
checksum
compatibility.

Keep previous working model where practical.

=====

161.MODEL ROLLBACK

=====

If new model performs worse:

rollback.

=====

162.MODEL BENCHMARK

=====

Evaluate:

latency
memory
quality
stability.

=====

163.HUMAN EVALUATION

=====

Human review is mandatory.

Inspect:

dialogue
action
landscape
crowd
close-up
text

subtitles
fast cuts
dark scenes
bright scenes.

=====

164.A/B FRAMING

=====

Compare:

Original
Static Crop
Smart Crop.

Do not assume smart is always better.

=====

165.FRAME QUALITY

=====

Metrics:

subject retention
text retention
subtitle retention
crop severity
movement smoothness
human preference.

=====

166.RELEASE GATE

=====

CineMorph cannot be declared ready if:

playback is unstable
AI causes lag
audio drifts
framing jitters
theater hides content
resume fails
large files crash
unsupported files crash.

=====

167.THEATER UX GUARDIAN

=====

The user should feel:

anticipation
transition
immersion
continuity
focus.

Not:

loading frustration
technical confusion
UI clutter
constant AI movement.

=====

168.WAITING STRATEGY

=====

Use ticket/curtain/intro time to prepare.

But never fake processing status.

=====

169.PERCEIVED PERFORMANCE

=====

The system may begin:

basic playback
while AI prepares.

This is preferable to blocking the user.

=====

170.PROGRESSIVE ENHANCEMENT

=====

START:

safe framing.

THEN:

tracking.

THEN:

smart composition.

THEN:

advanced effects.

Never reverse the priority.

=====
171.AI DISABLE
=====

User may disable intelligent framing.

Movie must still work.

=====
172.PERFORMANCE SETTINGS
=====

Provide simple:

AUTO
PERFORMANCE
CINEMATIC
SAFE.

Advanced settings may exist separately.

=====
173.NO TECHNICAL OVERLOAD
=====

Do not force users to select:

model versions
worker counts
sampling rates.

=====

174.ACCESSIBILITY

=====

Theater controls require:

keyboard access
focus states
labels
contrast.

=====

175.MOTION REDUCTION

=====

Respect:

prefers-reduced-motion.

Reduce:

curtain animation
ticket animation
ambient effects.

Do not reduce actual video playback.

=====

176.AUDIO ACCESSIBILITY

=====

Provide:

mute
volume
track selection
subtitle selection where available.

=====

177.COLOR ACCESSIBILITY

=====

Do not communicate essential state only through color.

=====

178.ERROR UX

=====

Errors should be concise.

Examples:

"File format not supported by this browser."

"Smart framing is unavailable; continuing with safe framing."

"Enhanced audio unavailable; using original audio."

=====

179.NO FALSE CINEMA CLAIMS

=====

Never claim:

certified IMAX
Dolby Atmos
true theater acoustics
physical projection
professional cinema hardware.

Use:

IMAX-style presentation
cinematic mode
theater-inspired experience

where appropriate.

=====

180.NO CONTENT INVENTION

=====

Do not generate replacement frames merely to fill an aspect ratio unless the user explicitly enables an experimental generative mode in a future version.

Current CineMorph is presentation/reframing oriented.

=====

181.GENERATIVE AI BOUNDARY

=====

Generative frame expansion is NOT part of the core system.

If introduced later:

it must be clearly marked experimental.

=====

182.NO STORY ALTERATION

=====

CineMorph must never:

remove a key character
hide a critical object
alter dialogue context
change scene meaning

merely to satisfy aspect ratio.

=====

183.SOURCE PRESERVATION

=====

Original mode must remain the reference.

=====

184.RECOVERY

=====

If intelligent framing fails mid-movie:

continue using last safe frame.

=====

185.RECOVERY FROM STALE ANALYSIS

=====

If analysis arrives late:

discard it if playback has moved beyond its useful timestamp.

=====
186.NO STALE FRAME CONTROL
=====

Old model output must never suddenly move the screen.

=====
187.FRAME COMMAND TIMESTAMP
=====

Analysis output should include:

media/session identity
timestamp/window
model version
algorithm version.

Reject mismatched output.

=====
188.SESSION ISOLATION
=====

Starting a new movie invalidates analysis from the previous movie.

=====
189.MODEL SESSION ISOLATION
=====

Do not allow model results from session A to modify session B.

=====
190.MULTIPLE WINDOWS
=====

Avoid cross-window session corruption.

=====
191.SHARED CORE
=====

Shared OMNISTREAM infrastructure may provide:

storage
media identity
playback
settings
diagnostics.

CineMorph owns:

theater
framing
AI
cinematic presentation.

=====

192.ARCHITECTURE

=====

Conceptual:

CineMorph UI
↓
Cinema Controller
↓
Playback Controller
↓
Media Engine

and separately:

Cinema Controller
↓
Framing Engine
↓
Model Router
↓
Vision Models

and:

Cinema Controller
↓

Theater Renderer.

=====

193.NO UI MODEL LOGIC

=====

React/UI components must not directly execute complex vision logic.

=====

194.WORKER COMMUNICATION

=====

Use structured messages.

Avoid uncontrolled shared state.

=====

195.RENDERING ISOLATION

=====

Theater renderer should not depend on model implementation details.

=====

196.MODEL REPLACEMENT

=====

A model can be replaced without redesigning the theater UI.

=====

197.AUDIO REPLACEMENT

=====

Audio processing can be replaced without changing playback identity.

=====

198.FUTURE MODE

=====

Future cinematic modes may include:

2.39
custom
VR-like
ambient

multi-screen.

Do not add until core modes are excellent.

=====

199.FUTURE RESEARCH

=====

Potential future:

scene semantics
character identity
narrative context
camera intent
cinematic style recognition
better composition scoring
learned framing models.

=====

200.CUSTOM FRAMING MODEL

=====

If eventually trained:

dataset:
licensed/open/user-authorized.

labels:

preferred crop
subject importance
safe areas
composition quality.

=====

201.TRAINING PRINCIPLE

=====

Do not train a model merely to replicate an arbitrary aesthetic.

Define measurable objectives.

=====

202.MODEL DATA PRIVACY

=====

User local videos must not automatically become training data.

=====

203.MODEL LICENSE

=====

Verify model/license compatibility before distribution.

=====

204.PERFORMANCE TARGET

=====

The target is not:

maximum AI.

The target is:

best perceived cinematic experience within available hardware.

=====

205.ADAPTIVE ANALYSIS

=====

High-end device:

more intelligence.

Low-end device:

less intelligence.

Experience remains functional.

=====

206.HARDWARE DETECTION

=====

Use browser/device capability signals where available.

Do not collect unnecessary hardware fingerprinting data.

=====

207.THERMAL

=====

Long movies should not sustain unnecessary maximum inference.

=====

208.BATTERY

=====

Battery-saving mode should reduce optional workloads.

=====

209.GPU

=====

Use GPU acceleration when available.

Fallback safely.

=====

210.WEBGPU

=====

Optional.

Never required for basic CineMorph playback.

=====

211.WASM

=====

Use where measurable benefit exists.

=====

212.NO MAIN-THREAD BLOCKING

=====

Heavy work must not freeze:

controls
seek
pause
fullscreen

navigation.

=====

213.LONG MOVIE RULE

=====

A 3-hour movie should be treated as:

a stream of manageable windows,

not:

one enormous analysis job.

=====

214.ANALYSIS WINDOW

=====

Use short windows around playback.

=====

215.CACHE WINDOW

=====

Cache only useful near-future information.

=====

216.PREFETCH

=====

Analyze slightly ahead where useful.

Do not analyze excessively far ahead.

=====

217.QUALITY DEGRADATION

=====

If analysis falls behind:

reduce analysis rate.

Never allow backlog to grow indefinitely.

=====

218.BACKPRESSURE

=====

When analysis queue grows:

drop obsolete tasks.

=====

219.PLAYBACK PRIORITY

=====

The player is always real-time.

AI is best-effort.

=====

220.REAL-TIME DEFINITION

=====

"Real-time" means sufficiently synchronized with playback for the user experience.

It does NOT require:

every frame analyzed instantly.

=====

221.FRAME RATE STRATEGY

=====

Sample intelligently.

Use:

scene changes
motion
subject movement

to decide when additional analysis is necessary.

=====

222.STATIC HOLD

=====

If the best frame remains valid:

reuse it.

=====

223.MOTION TRACK

=====

When subject moves smoothly:

track instead of repeatedly running full detection.

=====

224.REDETECTION

=====

Run expensive detection when tracking confidence falls.

=====

225.DETECTION FREQUENCY

=====

Adaptive.

Not fixed unnecessarily.

=====

226.TRACK FAILURE

=====

If tracking fails:

re-detect.

If re-detection fails:

safe frame.

=====

227.MULTIPLE SUBJECTS

=====

Use group framing when multiple subjects are equally important.

=====

228.SINGLE SUBJECT

=====

Prefer appropriate lookroom rather than mechanical center.

=====

229.NO SUBJECT

=====

Use scene composition.

=====

230.LANDSCAPE

=====

Preserve:

horizon
major geometry
visual balance.

=====

231.ACTION

=====

Prioritize:

motion
main subject
direction.

=====

232.DIALOGUE

=====

Prioritize:

faces
eye-line
two-shot composition.

=====

233.HORROR/SUSPENSE

=====

Do not automatically center subject.

Preserve intentional negative space.

=====

234.COMEDY

=====

Preserve multi-character interaction.

=====

235.CREDITS

=====

Stable presentation.

=====

236.TITLE CARDS

=====

Protect text.

=====

237.FOREIGN TEXT

=====

Do not crop important signage merely to center faces.

=====

238.SUBTITLE REGION

=====

Protect bottom region.

=====

239.WATERMARK

=====

Do not intentionally remove provider/creator watermarks.

=====

240.CONTENT INTEGRITY

=====

CineMorph should never intentionally obscure source attribution/watermarks.

=====

241.POST-SHOW

=====

At end:

curtain

ticket

resume state.

Do not automatically start another movie unless user explicitly enabled such behavior.

=====

242.REWATCH

=====

Offer:

REWATCH

RETURN

possibly CONTINUE if incomplete.

=====

243.SESSIION COMPLETE

=====

Release:

workers

buffers

object URLs

audio resources

temporary analysis.

=====

244.TESTING

=====

Mandatory:

functional
visual
performance
media
AI
audio
security
long-duration.

=====
245.TEST CONTENT
=====

Include:

faces
multiple people
landscape
action
text
subtitles
credits
fast cuts
dark scenes
bright scenes
camera movement
letterboxed content
different aspect ratios.

=====
246.VISUAL VULTURE TEST
=====

Inspect:

curtain alignment
screen aperture
chair placement
ticket size
ticket animation
screen edges
video crop
controls
fullscreen

responsive geometry
ambient effects.

=====

247.PERFORMANCE VULTURE TEST

=====

Look for:

FPS drops
memory growth
CPU spikes
GPU overload
analysis backlog
audio drift.

=====

248.FAILURE VULTURE TEST

=====

Break:

model
audio enhancement
network
file
storage
fullscreen
worker.

The movie should continue whenever possible.

=====

249.FINAL RELEASE GATE

=====

CineMorph may be declared complete only if:

LOCAL PLAYBACK WORKS
SUPPORTED ONLINE PLAYBACK WORKS WHERE POSSIBLE
THEATER WORKS
1.90 WORKS
1.43 WORKS
ORIGINAL WORKS

SMART FRAMING WORKS OR FALLS BACK
AUDIO WORKS OR FALLS BACK
SUBTITLES WORK OR FALL BACK
TICKETS WORK
RESUME WORKS
LONG MOVIES REMAIN STABLE
AI DOES NOT BLOCK PLAYBACK
NO CRITICAL MEMORY LEAK
NO FABRICATED DATA
NO FALSE CAPABILITY CLAIMS
SECURITY REVIEW PASSED
VISUAL REVIEW PASSED
PERFORMANCE REVIEW PASSED.

=====

250.FINAL CINEMORPH PRINCIPLE

=====

CINEMORPH SHOULD FEEL LIKE:

THE USER ENTERED A THEATER.

NOT:

THE USER OPENED A WEB APP WITH A VIDEO AND SOME EFFECTS.

The theater must disappear into the experience.

The movie must remain the focus.

The AI must become invisible.

The transitions must feel natural.

The framing must feel intentional.

The audio must remain synchronized.

The interface must remain quiet.

The system must degrade gracefully.

The source must remain respected.

FINAL RULE:

WHEN IN DOUBT:

PRESERVE THE MOVIE.

PRESERVE THE FRAME.

PRESERVE THE AUDIO.

PRESERVE THE USER'S CONTROL.

PRESERVE PERFORMANCE.

PRESERVE TRUTH.

[END_OMNISTREAM_DOCUMENT_03]

[OMNISTREAM_DOCUMENT_04]

[NAME=UI_UX_ARCHITECTURE_SECURITY_QUALITY_GUARDIAN]

[TYPE=DESIGN+ARCHITECTURE+SECURITY+PERFORMANCE+QA_GUARDIAN]

[PRIORITY=HIGH]

[SCOPE=ENTIRE_OMNISTREAM]

[STATUS=FOUNDATIONAL]

PURPOSE:

This document is the permanent guardian for how OMNISTREAM should look,feel,behave,be structured,perform,and protect its users. It prevents future changes from producing a visually impressive but technically weak product,or a technically sophisticated system with poor usability.

CORE RULE:

THE USER SHOULD EXPERIENCE SIMPLICITY.

THE ARCHITECTURE SHOULD CONTAIN THE COMPLEXITY.

=====

01.DESIGN PHILOSOPHY

=====

OMNISTREAM contains two intentionally different experiences.

U-TUBE:

modern

clean

bright
video-centric
familiar
efficient.

CINEMORPH:
cinematic
theatrical
immersive
film-inspired
warm
dramatic.

They may share:

typography foundations
accessibility principles
interaction conventions
technical components.

They must NOT feel visually identical.

=====

02.MAIN LANDING

=====

The first screen is the OMNISTREAM gateway.

Purpose:

explain the two experiences immediately.

Primary choices:

U-TUBE
CINEMORPH.

The landing page should feel:

minimal
professional
premium
clear.

Use a bento/minimalist composition where appropriate.

Do not overload the first screen with technical information.

=====

03.U-TUBE LANDING

=====

U-Tube should communicate:

DISCOVER
SEARCH
WATCH
SUBSCRIBE.

Visual language:

white
red
clean surfaces
modern cards
clear hierarchy.

Avoid unnecessary cinematic effects.

=====

04.CINEMORPH LANDING

=====

CineMorph should communicate:

ENTER CINEMA
SELECT YOUR MEDIA
EXPERIENCE THE SCREEN.

Visual language:

paper
film
ticket
camera
reel
popcorn
theater

curtain
cinema typography.

Decorative elements should support the concept rather than dominate it.

=====

05.CINEMORPH SOURCE CHOICE

=====

Clearly present:

LOCAL FILE
ONLINE VIDEO.

Local file should receive visual priority because CineMorph is local-first.

=====

06.ADAPTIVE DESIGN

=====

Every page must respond to:

viewport width
height
orientation
fullscreen
DPR
browser capabilities.

Never design only for one screenshot size.

=====

07.RESPONSIVE BREAKPOINTS

=====

Do not blindly copy arbitrary breakpoint values.

Choose breakpoints based on:

content collapse
navigation usability
card readability
player geometry.

08.NO FIXED-CANVAS UI

Avoid hardcoded positioning such as:

left:500px
top:300px

for major layout.

Use:

flex
grid
container queries
relative geometry
responsive calculations.

09.U-TUBE NAVIGATION

Primary navigation:

HOME
SEARCH
SUBSCRIPTIONS
HISTORY
COLLECTIONS.

Optional:

SETTINGS.

Navigation must remain predictable.

10.CINEMORPH NAVIGATION

CineMorph should reduce navigation during playback.

Primary interaction becomes:

PLAYER
MODE
AUDIO
SUBTITLE
SETTINGS.

The movie must dominate the screen.

=====

11.INFORMATION HIERARCHY

=====

Every screen must have:

PRIMARY
SECONDARY
TERTIARY

information.

Never make every element visually equal.

=====

12.VISUAL HIERARCHY

=====

Primary content:

largest visual emphasis.

Secondary:

supporting information.

Controls:

visible when needed.

Decor:

lowest priority.

=====

13.NO UI OVERLOAD

Do not add:

- too many buttons
- too many badges
- too many cards
- too many animations
- too many colors
- too many settings.

14.BUTTON RULE

Every button must have:

- purpose
- action
- feedback
- disabled state if needed
- error behavior.

If no meaningful action exists:

do not render it.

15.NO DEAD BUTTONS

Never ship buttons that:

- do nothing
- show fake progress
- open empty dialogs
- pretend functionality exists.

16.LOADING UX

Every asynchronous operation should have an appropriate state:

LOADING
SUCCESS
EMPTY
ERROR.

Never leave the user wondering whether the application froze.

=====
17.LOADING MUST BE REAL
=====

Do not fabricate percentages.

If actual progress is unknown:

use indeterminate progress.

=====
18.PERCEIVED PERFORMANCE
=====

Use meaningful parallel work.

Example:

CineMorph:

ticket animation
+
metadata extraction
+
model initialization
+
initial analysis.

Do not fake processing.

=====
19.SKELETON UI
=====

Skeletons are allowed for:

video cards
metadata
feed.

They must represent actual loading areas.

=====

20.ERROR DESIGN

=====

Errors must be:

short
specific
actionable.

Avoid technical stack traces in user-facing UI.

=====

21.EMPTY STATES

=====

Empty state should answer:

WHAT IS EMPTY?

WHY?

WHAT CAN I DO?

=====

22.CONFIRMATION

=====

Destructive actions:

clear history
clear collections
reset data

require appropriate confirmation.

=====

23.UNDO

=====

Where practical:

support undo for reversible actions.

=====

24.TOASTS

=====

Use for:

small confirmations
nonblocking information.

Do not use to communicate critical errors.

=====

25.MODALS

=====

Use only when:

decision
confirmation
focused interaction

requires interruption.

=====

26.NO MODAL ABUSE

=====

Do not open dialogs for ordinary navigation.

=====

27.TYPOGRAPHY

=====

Typography must provide:

hierarchy
readability
consistent scale.

Avoid too many font families.

=====

28.COLOR

=====

U-Tube:

white
red
neutral gray/black.

CineMorph:

paper
warm cinematic tones
film-inspired accents.

Color must communicate state consistently.

=====

29.CONTRAST

=====

All important text and controls must remain readable.

=====

30.ICON

=====

Use consistent iconography.

Do not mix:

random icon libraries
emoji
unrelated visual styles.

=====

31.ANIMATION

=====

Animations should communicate:

state
transition
depth
feedback.

Not:

decoration for decoration's sake.

=====
32.ANIMATION PERFORMANCE
=====

Prefer:

transform
opacity.

Avoid expensive layout animation.

=====
33.REDUCED MOTION
=====

Respect:

prefers-reduced-motion.

=====
34.U-TUBE MOTION
=====

Subtle:

card hover
navigation
mini-player transition.

No excessive motion.

=====
35.CINEMORPH MOTION
=====

Cinematic:

curtain
ticket
screen
transition
ambient lighting.

But playback remains stable.

=====

36.THEATER ANIMATION

=====

Theater effects must never cover:

critical video
subtitles
controls.

=====

37.Z-INDEX

=====

Define predictable stacking layers.

Example:

background
theater
screen
video
ambient
controls
dialogs.

Avoid arbitrary z-index escalation.

=====

38.SCROLLING

=====

Avoid nested scroll containers unless justified.

39.FOCUS

Keyboard focus must always remain visible.

40.KEYBOARD NAVIGATION

Primary actions must be keyboard accessible.

41.SCREEN READERS

Interactive elements require meaningful accessible labels.

42.SEMANTIC HTML

Prefer:

button
nav
main
header
section
article

over generic divs.

43.ARIA

Use ARIA when necessary.

Do not use ARIA to compensate for poor HTML structure.

44.FORM INPUTS

=====

Search:

clear label
placeholder
keyboard support
validation.

=====

45.U-TUBE CARD ACCESSIBILITY

=====

Card should expose:

title
channel
duration
action.

=====

46.CINEMORPH CONTROL ACCESSIBILITY

=====

Controls:

mode
play
pause
volume
fullscreen
audio
subtitle

must remain accessible.

=====

47.FULLSCREEN UX

=====

Fullscreen must:

enter predictably
exit predictably

preserve state.

=====

48.PLAYER CONTROLS

=====

Controls should:

appear when needed
remain readable
avoid obstructing content.

=====

49.HIDE CONTROLS

=====

After inactivity:

controls may fade.

Interaction should bring them back.

=====

50.MINI PLAYER

=====

Mini-player:

persistent
compact
nonintrusive.

It must not block important UI.

=====

51.U-TUBE VISUAL QA

=====

Check:

cards
thumbnails
duration
progress

channel avatar
sidebar
search
mini-player
empty states
loading
errors.

=====

52.CINEMORPH VISUAL QA

=====

Check:

ticket
printer
curtains
screen
seats
screen aperture
video crop
controls
ambient effects
fullscreen.

=====

53.ARCHITECTURE PRINCIPLE

=====

Separate:

UI
APPLICATION
DOMAIN
SERVICES
ADAPTERS
INFRASTRUCTURE.

=====

54.UI LAYER

=====

UI handles:

presentation
interaction
visual state.

UI must not own:

provider logic
model inference
storage algorithms.

=====

55.APPLICATION LAYER

=====

Application layer coordinates:

user actions
services
state transitions.

=====

56.DOMAIN LAYER

=====

Domain defines:

media
playback
subscription
history
collection
cinema session
framing concepts.

=====

57.SERVICE LAYER

=====

Services perform:

search
storage
analysis
playback

model routing
audio.

=====

58.ADAPTER LAYER

=====

Adapters isolate:

YouTube
browser APIs
WebGPU
WASM
filesystem APIs
future providers.

=====

59.INFRASTRUCTURE

=====

Infrastructure manages:

storage
network
workers
model assets
configuration.

=====

60.SHARED CORE

=====

Shared core should contain only genuinely shared capabilities.

=====

61.NO GOD SERVICE

=====

Avoid one giant:

OmniStreamService.

Prefer focused services.

62.NO GOD COMPONENT

Avoid one giant React component controlling the entire application.

63.COMPONENT BOUNDARIES

Components should have:

one clear responsibility.

64.STATE MANAGEMENT

Global state only for truly global state.

Do not place every UI toggle in global state.

65.SERVER STATE

External provider data should be handled separately from local UI state.

66.LOCAL STATE

Use local state for:

menus
dialogs
hover
temporary input.

67.PERSISTENT STATE

Persist:

history
subscriptions
collections
preferences
tickets.

=====

68.SESSION STATE

=====

Keep:

active media
position
playback state
mode
audio
subtitle.

=====

69.STATE MACHINE

=====

Playback must use explicit state transitions.

=====

70.NO IMPOSSIBLE STATES

=====

Prevent:

PLAYING + ERROR.

LOADING + ENDED.

UNINITIALIZED + SEEKING.

=====

71.ASYNC RACE PROTECTION

=====

Latest valid action wins.

=====
72.CANCELLATION
=====

Cancel stale:

search
metadata
analysis
model operations.

=====
73.WORKERS
=====

Use workers for heavy processing.

=====
74.WORKER BOUNDARIES
=====

Workers receive:

input
configuration
session identity.

Workers return:

timestamped result
model version
algorithm version.

=====
75.STALE WORKER RESULT
=====

Reject if:

session changed
media changed
timestamp obsolete
algorithm incompatible.

=====

76.PERFORMANCE

=====

Primary metrics:

startup
interaction latency
playback smoothness
memory
CPU
GPU
network requests.

=====

77.PERFORMANCE BUDGET

=====

Every feature should have an approximate resource budget.

=====

78.NO PERFORMANCE REGRESSION

=====

Visual improvements must not silently create:

large bundles
slow startup
continuous CPU work.

=====

79.CODE SPLITTING

=====

Separate heavy CineMorph functionality from U-Tube.

=====

80.LAZY LOAD

=====

Load advanced:

models
theater assets

audio processors

only when required.

=====

81.PREFETCH

=====

Prefetch only when:

high confidence
low cost
meaningful benefit.

=====

82.CACHE

=====

Cache:

metadata
models
analysis
assets

with bounded size/versioning.

=====

83.CACHE INVALIDATION

=====

Invalidate based on:

version
media identity
algorithm version
expiration.

=====

84.NO UNBOUNDED CACHE

=====

Everything cached needs a cleanup policy.

=====

85.MEMORY

=====

Release:

workers
object URLs
VideoFrames
ImageBitmaps
audio nodes
event listeners.

=====

86.LONG SESSION

=====

Repeated use must not produce increasing memory consumption.

=====

87.SECURITY MODEL

=====

Trust boundaries:

USER INPUT
↓
VALIDATION
↓
APPLICATION
↓
EXTERNAL PROVIDER / LOCAL FILE / MODEL
↓
SANITIZED RESULT
↓
UI.

=====

88.UNTRUSTED INPUT

=====

Treat as hostile:

URLs

metadata
subtitles
filenames
provider responses
local files.

=====
89.XSS
=====

Never render untrusted HTML directly.

=====
90.CSP
=====

Use Content Security Policy where deployment supports it.

=====
91.IFRAME
=====

Restrict:

origins
permissions
navigation.

=====
92.CORS
=====

Never disable security as a shortcut.

=====
93.SECRETS
=====

No secrets in:

frontend bundle
Git
logs
local storage

public configuration.

=====

94.ENVIRONMENT

=====

Separate:

development
production.

=====

95.PRODUCTION DEBUG

=====

No:

debug overlays
test controls
fake data
verbose logs

in production unless explicitly gated.

=====

96.DEPENDENCY SECURITY

=====

Audit:

dependencies
versions
licenses
known vulnerabilities.

=====

97.MODEL SECURITY

=====

Verify:

source
checksum
version

license.

=====

98.MODEL PRIVACY

=====

Local media must not automatically leave the device.

=====

99.TELEMETRY

=====

No hidden tracking.

If analytics are ever added:

explicit
minimal
documented
privacy-aware.

=====

100.DATA MINIMIZATION

=====

Store only what the feature requires.

=====

101.LOCAL STORAGE SECURITY

=====

Never store secrets.

=====

102.FILE SECURITY

=====

Never execute local file content.

=====

103.MEDIA SECURITY

=====

Malformed media must not crash the application.

=====

104.RESOURCE EXHAUSTION

=====

Bound:

file size handling
analysis queue
model memory
cache
requests
retries.

=====

105.RATE LIMITING

=====

Protect provider requests.

=====

106.RETRY POLICY

=====

Bound retries.

Use exponential backoff where appropriate.

=====

107.NO INFINITE LOOPS

=====

No retry loop or polling loop may run indefinitely without explicit control.

=====

108.ERROR ISOLATION

=====

One subsystem failing must not crash the entire product.

=====

109.FALLBACK TREE

=====

U-TUBE:

provider

→cached

→empty/error.

CINEMORPH:

AI

→rules

→safe frame

→original.

AUDIO:

enhanced

→original.

THEATER:

advanced

→basic.

=====
110.NO FALSE FALLBACK
=====

Fallback must contain genuine data/functionality.

=====
111.TESTING LAYERS
=====

UNIT
INTEGRATION
E2E
VISUAL
PERFORMANCE
SECURITY
MANUAL.

=====
112.UNIT TESTS
=====

Test:

geometry
state
ranking
storage
migration
validation.

=====

113.INTEGRATION TESTS

=====

Test:

provider
storage
player
models
workers.

=====

114.E2E

=====

Test complete user journeys.

=====

115.VISUAL REGRESSION

=====

Compare important pages after major changes.

=====

116.PERFORMANCE TEST

=====

Measure:

startup
search
playback
CineMorph startup

AI warm-up
long movie.

=====

117.SECURITY TEST

=====

Test:

XSS
malformed URLs
malformed media
malicious subtitles
storage abuse
provider responses.

=====

118.MANUAL VULTURE PASS

=====

After automation:

inspect manually.

Look for:

tiny alignment errors
wrong text
broken transitions
clipped content
unexpected scrollbars
z-index bugs
dead buttons
inconsistent icons
wrong states.

=====

119.REGRESSION RULE

=====

Every shared-core change requires:

U-Tube test
AND

CineMorph test.

=====

120.BROWSER MATRIX

=====

Test supported major browsers.

Do not claim unsupported compatibility.

=====

121.DEVICE MATRIX

=====

Test:

high-end
mid-range
low-end.

=====

122.VIEWPORT MATRIX

=====

Test:

small
medium
large
ultrawide.

=====

123.MEDIA MATRIX

=====

Test:

short
long
low resolution
high resolution
different aspect ratios
different codecs where supported
multiple audio tracks

subtitles.

=====

124.NETWORK MATRIX

=====

Test:

fast
slow
offline
intermittent.

=====

125.USER FLOW MATRIX

=====

Test:

first launch
returning user
resume
abandon
reset
error recovery.

=====

126.DATA MIGRATION

=====

Version persisted data.

Never assume old schema is current.

=====

127.MIGRATION SAFETY

=====

Migration:

detect
backup where practical
transform
validate

commit.

=====

128.CORRUPTION RECOVERY

=====

If state is corrupted:

recover safe subset.

Do not crash startup.

=====

129.NO DATA LOSS

=====

A feature change must never casually delete:

history
subscriptions
collections
tickets.

=====

130.RELEASE PROCESS

=====

INSPECT
→PLAN
→IMPLEMENT
→TEST
→REVIEW
→OPTIMIZE
→SECURE
→RETEST
→RELEASE.

=====

131.NO BLIND IMPLEMENTATION

=====

Never modify code merely because an AI model suggested it.

Understand:

why
what
risk
test.

=====

132.NO BLIND REWRITE

=====

Do not replace functioning architecture without measurable justification.

=====

133.NO PREMATURE ENTERPRISE

=====

Personal project:

do not introduce unnecessary:

microservices
Kubernetes
distributed databases
complex authentication.

=====

134.MODULARITY

=====

Design modules so future industrialization is possible.

Do not build industrial infrastructure prematurely.

=====

135.PROVIDER ADAPTER

=====

Provider-specific logic remains isolated.

=====

136.MODEL ADAPTER

=====

Model-specific logic remains replaceable.

=====

137.AUDIO ADAPTER

=====

Audio processors remain replaceable.

=====

138.RENDERER ADAPTER

=====

Theater renderer can evolve without rewriting media identity.

=====

139.CONFIGURATION

=====

Separate:

product defaults
user settings
environment config
developer settings.

=====

140.NO HARDCODED SECRETS

=====

Never commit credentials.

=====

141.NO HARD-CODED DEVICE GEOMETRY

=====

Never assume:

1920×1080.

=====

142.NO HARD-CODED MODEL ASSUMPTION

=====

Model availability must be checked.

=====

143.NO HARD-CODED PROVIDER SUCCESS

=====

Provider calls may fail.

=====

144.NO HARD-CODED CODEC SUPPORT

=====

Detect capability.

=====

145.NO HARD-CODED FULLSCREEN

=====

Fullscreen can fail.

=====

146.NO HARD-CODED AUDIO

=====

Track availability varies.

=====

147.NO HARD-CODED SUBTITLE

=====

Subtitle availability varies.

=====

148.NO HARD-CODED FILE ACCESS

=====

Browser permissions vary.

=====

149.ACCESSIBILITY REGRESSION

=====

Visual changes must not remove:

keyboard access
focus

labels
contrast.

=====

150.REDUCED MOTION REGRESSION

=====

New animations must respect motion preferences.

=====

151.PERFORMANCE REGRESSION

=====

New visual effects must be benchmarked.

=====

152.SECURITY REGRESSION

=====

New external dependency must trigger security review.

=====

153.PRIVACY REGRESSION

=====

New external service must trigger privacy review.

=====

154.COST REGRESSION

=====

New cloud/API/model dependency must document:

cost
quota
fallback.

=====

155.FREE-FIRST REGRESSION

=====

Core experience should remain usable without mandatory paid infrastructure.

=====

156.UI DATA TRUTH

=====

Every number displayed must have a source.

Examples:

runtime
views
progress
watched percentage.

=====

157.NO FAKE STATISTICS

=====

Never generate plausible-looking numbers.

=====

158.NO FAKE STATUS

=====

If model isn't loaded:

show not ready/loading.

Do not show:

"AI optimized."

=====

159.NO FAKE PROGRESS

=====

Use real progress or indeterminate state.

=====

160.NO FAKE DOWNLOAD

=====

No download button without genuine implementation.

=====

161.NO FAKE PREMIUM

=====

No artificial locked features.

=====

162.NO FAKE RECOMMENDATION

=====

No invented recommendation results.

=====

163.NO FAKE TICKET DATA

=====

Ticket must derive from actual session.

=====

164.NO FAKE PLAYER

=====

A visual player without functional playback is unacceptable.

=====

165.NO DECORATIVE CORE

=====

Visual effects cannot substitute missing functionality.

=====

166.PERFORMANCE PRIORITY

=====

If a design effect causes:

lag
memory growth
input delay

remove/reduce it.

=====

167.ANIMATION PRIORITY

=====

Use animation to:

explain.

Not:

delay.

=====
168.LOADING PRIORITY
=====

Whenever possible:

work while user sees meaningful content.

=====
169.PERCEIVED LATENCY
=====

Do useful preparation during:

ticket
intro
transition.

=====
170.NO ARTIFICIAL WAIT
=====

Do not intentionally delay playback merely to make the product appear sophisticated.

=====
171.ARCHITECTURAL DOCUMENTATION
=====

Maintain:

component map
service map
data flow
state flow
media pipeline
model pipeline

storage schema
security boundaries.

=====

172.DECISION LOG

=====

Record major architectural decisions.

=====

173.TECHNICAL DEBT

=====

Maintain debt list.

Do not hide known shortcuts.

=====

174.KNOWN LIMITATIONS

=====

Document:

browser
codec
provider
GPU
model
audio
fullscreen
storage.

=====

175.NO OVERPROMISING

=====

Never claim:

universal
instant
lossless
certified
perfect
100%

real IMAX
all formats

unless verified.

=====

176.QUALITY GATE

=====

Before declaring complete:

FUNCTIONAL
VISUAL
PERFORMANCE
SECURITY
PRIVACY
ACCESSIBILITY
RELIABILITY

must all be reviewed.

=====

177.VULTURE CHECK

=====

Inspect every major route:

landing
U-Tube
search
watch
subscriptions
history
collections
CineMorph landing
source selection
ticket
theater
settings
errors.

=====

178.VULTURE CHECK DETAILS

=====

Look for:

1px errors
misalignment
overflow
broken state
wrong hover
wrong cursor
wrong focus
unexpected scroll
incorrect responsive behavior
animation stutter
incorrect typography
inconsistent spacing.

=====

179.FINAL UX TEST

=====

Give the product to someone without explaining it.

Observe:

Can they understand it?

Can they search?

Can they watch?

Can they subscribe?

Can they enter CineMorph?

Can they resume?

If not:

simplify.

=====

180.FINAL ARCHITECTURAL TEST

=====

Ask:

Can U-Tube change without breaking CineMorph?

Can CineMorph change without breaking U-Tube?

Can a model be replaced?

Can a provider be replaced?

Can storage schema evolve?

Can advanced AI fail safely?

If not:

improve boundaries.

=====
181.FINAL SECURITY TEST
=====

Ask:

What can an attacker control?

What can an attacker inject?

What can an attacker upload?

What can an attacker force the browser to execute?

What secrets exist?

What data leaves the device?

Reduce every unnecessary attack surface.

=====
182.FINAL PERFORMANCE TEST
=====

Ask:

What happens after 3 hours?

What happens on a weak device?

What happens when AI is slow?

What happens when network is slow?

What happens when storage is full?

What happens when the browser sleeps?

The application must degrade gracefully.

=====
183.FINAL PRODUCT TEST
=====

Ask:

Does this feature make the experience better?

If yes:

keep.

If neutral:

question it.

If worse:

remove/rework.

=====
184.CHANGE AUTHORITY
=====

Any developer/agent may suggest changes.

No change should override the product constitution without explicit architectural justification.

=====
185.GUARDIAN PRIORITY

=====

When documents conflict:

PRODUCT MOTIVE

>

USER SAFETY

>

DATA INTEGRITY

>

CORE PLAYBACK

>

PERFORMANCE

>

UX

>

ADVANCED FEATURES

>

DECORATION.

=====

186.FINAL RULE

=====

OMNISTREAM MUST NEVER BECOME:

A TECH DEMO DISGUISED AS A PRODUCT.

It must remain:

A PRODUCT POWERED BY TECH.

=====

187.FINAL QUALITY STATEMENT

=====

The user should not need to understand:

providers

workers

models

caches

state machines

GPU

WASM
APIs
fallbacks
storage
framing algorithms.

They should only experience:

DISCOVER.
WATCH.
ENTER.
ENJOY.
RESUME.

=====
188.FINAL GUARDIAN STATEMENT
=====

WHENEVER A FUTURE CHANGE IMPROVES ONE DIMENSION BUT DAMAGES ANOTHER:

MEASURE THE TRADEOFF.

WHENEVER VISUAL BEAUTY DAMAGES PERFORMANCE:

PERFORMANCE WINS.

WHENEVER AI QUALITY DAMAGES PLAYBACK:

PLAYBACK WINS.

WHENEVER CONVENIENCE DAMAGES SECURITY:

SECURITY WINS.

WHENEVER COMPLEXITY DOES NOT CREATE USER VALUE:

SIMPLICITY WINS.

WHENEVER A FEATURE CANNOT BE MADE RELIABLE:

MAKE IT OPTIONAL OR REMOVE IT.

WHENEVER THE SYSTEM DOES NOT KNOW:

DO NOT INVENT.

WHENEVER THE PRODUCT FAILS:

FAIL GRACEFULLY.

FINAL PRINCIPLE:

OMNISTREAM SHOULD FEEL SIMPLE ON THE OUTSIDE BECAUSE IT IS DISCIPLINED ON THE INSIDE.

[END_OMNISTREAM_DOCUMENT_04]

[OMNISTREAM_DOCUMENT_05]

[NAME=OPERATIONAL_DEVELOPMENT_TESTING_FUTURE_GUARDIAN]

[TYPE=ENGINEERING+RELEASE+TESTING+MAINTENANCE+FUTURE_SCOPE_GUARDIAN]

[PRIORITY=HIGH]

[SCOPE=ENTIRE_OMNISTREAM]

[STATUS=FOUNDATIONAL]

PURPOSE:

This document governs how OMNISTREAM is developed,modified,tested,optimized,secured,released,maintained,and expanded. It exists to prevent uncontrolled feature growth,AI-generated dead code,architectural degradation,fake completion claims,security weaknesses,performance regressions,and unnecessary complexity.

CORE PRINCIPLE:

INSPECT → UNDERSTAND → PLAN → CHANGE → TEST → VERIFY → OPTIMIZE → RELEASE.

NEVER:

GUESS → MODIFY → CLAIM COMPLETE.

=====

01.PROJECT OPERATING PRINCIPLE

=====

OMNISTREAM is a personal/free-first project.

The current objective is to build an excellent usable product,not an enterprise-scale commercial platform.

Future industrialization is possible.

Industrial architecture must NOT be introduced prematurely.

=====

02.DEVELOPMENT PRIORITY

=====

P0:
CORE FUNCTIONALITY
SECURITY
DATA INTEGRITY
PLAYBACK
STABILITY.

P1:
UX
PERFORMANCE
RESPONSIVENESS
ACCESSIBILITY.

P2:
INTELLIGENCE
AUTOMATION
ADVANCED CINEMATIC FEATURES.

P3:
EXPERIMENTAL/FUTURE FEATURES.

=====

03.BEFORE ANY CHANGE

=====

FIRST:

inspect repository.

inspect existing architecture.

inspect package configuration.

inspect routes.

inspect components.

inspect services.

inspect storage.

inspect environment variables.

inspect provider integrations.

inspect workers.

inspect model assets.

inspect build scripts.

inspect deployment configuration.

inspect tests.

inspect known issues.

=====

04.NO BLIND REWRITE

=====

Never rewrite functioning code simply because:

another architecture looks cleaner,
an AI suggested it,
a framework is newer,
the code could theoretically be shorter.

Change only when measurable benefit exists.

=====

05.EXISTING FUNCTION PROTECTION

=====

Before changing a shared component:

identify all consumers.

Before changing a shared service:

identify all dependencies.

Before changing storage:

identify all persisted data.

Before changing playback:

test both U-Tube and CineMorph.

=====

06.CHANGE CLASSIFICATION

=====

Every change should be classified:

BUGFIX
PERFORMANCE
SECURITY
UX
FEATURE
REFACTOR
EXPERIMENTAL.

=====

07.CHANGE RISK

=====

LOW:

isolated UI adjustment.

MEDIUM:

shared component/service change.

HIGH:

playback
storage
authentication
provider
model

worker
security.

HIGH-RISK changes require regression testing.

=====
08.BEFORE FEATURE ADDITION
=====

Ask:

What problem does this solve?

Who benefits?

How often?

What does it cost?

What does it complicate?

What happens if it fails?

Can existing functionality solve it?

=====
09.FEATURE ACCEPTANCE
=====

A feature is justified only if:

USER VALUE
>
COMPLEXITY COST.

=====
10.NO FEATURE COUNTING
=====

More features does not mean better product.

=====
11.NO AI FEATURE THEATER
=====

Do not add AI because:

"AI is modern."

AI must solve a measurable problem.

=====

12.AI DECISION TEST

=====

Ask:

Can rules solve it?

Can a small model solve it?

Can an existing model solve it?

Can hybrid logic solve it?

Only then consider custom training.

=====

13.CUSTOM MODEL

=====

Build/train a custom model only when:

existing solutions fail measurable requirements.

=====

14.MODEL OBJECTIVE

=====

Define:

input
output
metric
latency target
memory target
failure behavior.

=====

15.MODEL TRAINING

=====

Training data must be:

legal/licensed/authorized.

No automatic training on user media.

=====

16.MODEL EVALUATION

=====

Evaluate:

accuracy
latency
memory
stability
false positives
false negatives.

=====

17.MODEL RELEASE

=====

A model is not production-ready because it works once.

Test across diverse content.

=====

18.MODEL VERSIONING

=====

Version:

model
dataset
preprocessing
postprocessing.

=====

19.MODEL ROLLBACK

=====

Maintain ability to revert to previous stable version.

=====

20.MODEL FAILURE

=====

Every model must have fallback.

=====

21.LLM POLICY

=====

LLMs may assist:

- query interpretation
- metadata classification
- developer workflows
- documentation
- high-level semantic analysis.

LLMs must NOT control:

- frame-by-frame rendering
- real-time playback
- critical security
- unvalidated external actions.

=====

22.LLM PRODUCTIVITY LAYER

=====

A future internal LLM assistant may provide:

- project inspection
- error explanation
- architecture suggestions
- test generation
- documentation generation
- performance analysis
- code search
- dependency analysis.

It must operate as an assistant, not autonomous authority.

=====

23.LLM SAFETY

=====

AI-generated changes must be:

reviewed
tested
validated.

Never allow an LLM to silently modify production architecture.

=====

24.AUTONOMOUS AGENT POLICY

=====

An agent may:

inspect
suggest
implement
test.

But before destructive operations:

protect data
create rollback point
verify scope.

=====

25.AGENT DECISION RULE

=====

If user explicitly delegates a decision:

choose the option that best benefits:

reliability
performance
security
UX
maintainability.

Never optimize for novelty.

=====

26.ARCHITECTURE LAYERS

=====

Recommended conceptual structure:

PRESENTATION
↓
APPLICATION
↓
DOMAIN
↓
SERVICES
↓
ADAPTERS
↓
INFRASTRUCTURE.

=====

27.U-TUBE BOUNDARY

=====

U-Tube owns:

search
subscription
feed
history
collections
online playback UX.

=====

28.CINEMORPH BOUNDARY

=====

CineMorph owns:

local media
theater
framing
AI analysis

cinematic presentation
audio enhancement.

=====

29.SHARED CORE

=====

Shared:

media identity
playback state
storage utilities
settings
diagnostics
common UI primitives.

=====

30.NO CROSS-CONTAMINATION

=====

U-Tube code should not depend unnecessarily on CineMorph models.

CineMorph should not require U-Tube recommendation systems.

=====

31.PROVIDER ABSTRACTION

=====

Provider-specific behavior belongs behind adapters.

=====

32.MODEL ABSTRACTION

=====

Model-specific implementation belongs behind model interfaces.

=====

33.AUDIO ABSTRACTION

=====

Audio processing belongs behind audio interfaces.

=====

34.RENDERER ABSTRACTION

=====

Theater renderer should remain replaceable.

=====

35.STORAGE ABSTRACTION

=====

Application logic should not depend directly on low-level storage APIs everywhere.

=====

36.DATA MODELS

=====

Core entities may include:

Video
Channel
Subscription
HistoryEntry
Collection
PlaybackSession
CinemaSession
Ticket
MediaSource
AudioTrack
SubtitleTrack
ModelResult.

=====

37.STABLE IDENTIFIERS

=====

Use stable IDs.

Never rely on display titles as primary identity.

=====

38.SCHEMA VERSION

=====

Persisted data requires schema version.

=====

39.MIGRATION

Every schema change requires migration logic.

40.MIGRATION TEST

Test:

old data

→migration

→new data

→read/write.

41.DATA BACKUP

Before risky migrations:

preserve recoverable state where practical.

42.NO DATA LOSS

Never delete user data during routine updates.

43.RESET

Reset functionality must clearly state what will be deleted.

44.LOCAL-FIRST

Prefer local processing for:

history

subscriptions

collections
tickets
local media
CineMorph analysis.

=====
45.EXTERNAL SERVICES
=====

External services should be introduced only when necessary.

=====
46.COST GUARDIAN
=====

Every external dependency must document:

price
free tier
quota
rate limit
failure behavior.

=====
47.FREE-FIRST
=====

Core functionality should remain usable without mandatory paid services wherever technically feasible.

=====
48.NO UNLIMITED CLAIM
=====

Free services may have:

quotas
rate limits
storage limits
model limits.

Document them.

=====

49.COST FALLBACK

=====

If an external service becomes unavailable:

fallback to:

local
cached
deterministic
manual.

=====

50.SECURITY FIRST

=====

Security is not a final-stage feature.

It is part of architecture.

=====

51.THREAT MODEL

=====

Consider:

malicious URLs
malicious metadata
malicious subtitles
malformed media
XSS
injection
dependency compromise
credential exposure
resource exhaustion
storage abuse
model poisoning.

=====

52.INPUT VALIDATION

=====

Validate every external input.

=====

53.OUTPUT SANITIZATION

=====

Sanitize data before UI rendering.

=====

54.SECRET MANAGEMENT

=====

Secrets must never exist in:

source
public frontend bundle
Git
local storage.

=====

55.ENVIRONMENT VARIABLES

=====

Separate development and production configuration.

=====

56.SECURITY HEADERS

=====

Where deployment allows:

CSP
HSTS
frame restrictions
content-type protection
referrer policy.

=====

57.IFRAME POLICY

=====

Only allow known legitimate origins.

=====

58.EXTERNAL SCRIPT POLICY

=====

Minimize third-party scripts.

=====

59.DEPENDENCY POLICY

=====

Every dependency should have a reason.

=====

60.DEPENDENCY AUDIT

=====

Regularly inspect:

vulnerabilities
licenses
abandonment
bundle size.

=====

61.DEAD CODE

=====

Remove:

unused imports
unused functions
unused components
obsolete utilities
dead feature flags.

=====

62.DEAD DEPENDENCIES

=====

Remove unused packages.

=====

63.DUPLICATE LOGIC

=====

Consolidate duplicated logic only after confirming behavior is equivalent.

=====

64.CODE QUALITY

=====

Prefer:

clear naming
small modules
predictable state
explicit contracts.

=====

65.NO OVERABSTRACTION

=====

Do not create abstractions before repeated complexity justifies them.

=====

66.NO GOD FILES

=====

Break extremely large files when responsibility boundaries become unclear.

=====

67.NO GOD CLASSES

=====

Same principle for classes/services.

=====

68.NO MAGIC VALUES

=====

Centralize meaningful configuration.

=====

69.TYPE SAFETY

=====

Use strong typing where supported.

=====

70.RUNTIME VALIDATION

=====

Type correctness does not replace runtime validation.

=====
71.ERROR HANDLING
=====

Every async boundary must have:

success
failure
cancellation.

=====
72.ERROR BOUNDARIES
=====

A failure in one subsystem should not crash the application.

=====
73.USER ERROR VS SYSTEM ERROR
=====

Distinguish:

invalid input
unsupported capability
network failure
provider failure
internal failure.

=====
74.LOGGING
=====

Logs should support debugging without leaking sensitive data.

=====
75.LOG LEVELS
=====

Use:

debug

info
warn
error.

Production should avoid noisy debug logging.

=====
76.ERROR MONITORING
=====

If future telemetry is introduced:

privacy-first
minimal
documented.

=====
77.NO HIDDEN TELEMETRY
=====

Do not silently track users.

=====
78.PERFORMANCE BUDGET
=====

Track:

bundle size
startup
interaction latency
memory
CPU
GPU
network.

=====
79.BUNDLE OPTIMIZATION
=====

Use:

code splitting
tree shaking

lazy loading.

=====

80.IMAGE OPTIMIZATION

=====

Use:

proper sizing
lazy loading
modern formats where supported.

=====

81.VIDEO OPTIMIZATION

=====

Do not load unnecessary media.

=====

82.MODEL OPTIMIZATION

=====

Use:

quantization
smaller models
lower resolution
adaptive inference

when quality remains acceptable.

=====

83.CINEMORPH PERFORMANCE

=====

Priority:

PLAYBACK
→AUDIO
→RENDERING
→INPUT
→AI
→EFFECTS.

=====

84.U-TUBE PERFORMANCE

=====

Priority:

SEARCH
→PLAYER
→SCROLLING
→FEED
→RECOMMENDATION.

=====

85.BACKPRESSURE

=====

Any queue that can grow must have:

maximum size
drop strategy
cancellation.

=====

86.REQUEST CONTROL

=====

Use:

debounce
cache
deduplication
abort
backoff.

=====

87.RETRY LIMIT

=====

No infinite retries.

=====

88.POLLING

=====

Every polling system requires:

interval
stop condition
visibility handling
error backoff.

=====
89.BACKGROUND WORK
=====

Pause/deprioritize unnecessary work when application is hidden.

=====
90.VISIBILITY
=====

Use browser visibility signals where appropriate.

=====
91.RESOURCE CLEANUP
=====

Always release:

listeners
workers
timers
object URLs
media resources
temporary buffers.

=====
92.LONG-SESSION TEST
=====

Test:

30 min
1 hour
2 hours
3 hours.

=====

93.MEMORY LEAK TEST

=====

Compare memory:

start
30m
1h
2h
3h.

Memory should not continuously grow without reason.

=====

94.PLAYBACK TEST

=====

Test:

play
pause
seek
resume
fullscreen
volume
mode change
audio change
subtitle change.

=====

95.LOCAL MEDIA TEST

=====

Test:

short video
3-hour video
high resolution
low resolution
large file
multiple tracks
subtitles
different aspect ratios.

=====

96.ONLINE MEDIA TEST

=====

Test:

valid
invalid
removed
restricted
unsupported
slow network
offline.

=====

97.U-TUBE TEST

=====

Search.

Select.

Watch.

Mini player.

History.

Subscription.

Feed.

Collections.

=====

98.CINEMORPH TEST

=====

Landing.

Source selection.

Ticket.

Intro.

Theater.

Mode.

Framing.

Audio.

Subtitle.

Resume.

Exit.

=====

99.THEATER TEST

=====

Test:

small screen
large screen
fullscreen
ultrawide
different aspect ratios.

=====

100.FRAME TEST

=====

Test:

single face
multiple faces
no faces
fast action
dialogue
landscape
text
subtitles
credits
dark scene

bright scene.

=====

101.FRAME FAILURE

=====

Force:

model unavailable
model timeout
worker failure
bad detection
stale detection.

Verify:

safe framing.

=====

102.AUDIO TEST

=====

Test:

original
enhanced
multiple tracks
switching
mute
volume
long playback.

=====

103.AUDIO FAILURE

=====

Force enhancement failure.

Verify:

original audio.

=====

104.FILE FAILURE

=====

Test:

corrupted
unsupported
missing
permission denied.

=====

105.NETWORK FAILURE

=====

Test:

disconnect during search
disconnect during playback
disconnect during refresh.

=====

106.STORAGE FAILURE

=====

Test:

quota
corrupt state
blocked storage.

=====

107.BROWSER FAILURE

=====

Test:

unsupported API
fullscreen unavailable
WebGPU unavailable
codec unavailable.

=====

108.FALLBACK TEST

=====

Every advanced capability must be deliberately disabled during testing.

=====

109.ACCESSIBILITY TEST

=====

Test:

keyboard
screen reader
contrast
focus
reduced motion.

=====

110.RESPONSIVE TEST

=====

Test:

mobile U-Tube
tablet
desktop
large desktop
ultrawide CineMorph.

=====

111.VISUAL TEST

=====

Inspect:

alignment
spacing
typography
color
icons
buttons
cards
transitions
theater layers.

=====

112.VULTURE REVIEW

=====

After functional testing:

zoom into the interface.

Look for:

tiny spacing errors

1px offsets

incorrect border radius

wrong font weight

misaligned icons

clipped text

unexpected scrollbars

broken hover

bad focus

z-index errors.

=====

113.USER FLOW TEST

=====

First-time user:

landing

→U-Tube

→search

→watch.

Second:

landing

→CineMorph

→local file

→ticket

→theater.

Third:

resume previous session.

No explanation should be required.

=====

114.REGRESSION TEST

=====

After every significant change:

run affected tests.

After shared-core change:

run both product tracks.

=====

115.BUILD TEST

=====

Production build must complete without critical errors.

=====

116.CONSOLE TEST

=====

Production should have:

no unexpected errors
no repeated warnings
no leaked secrets.

=====

117.NETWORK TEST

=====

Inspect:

unexpected requests
duplicate requests
unnecessary polling
large payloads.

=====

118.SECURITY TEST

=====

Test:

input injection
XSS
URL manipulation
malicious metadata
malicious subtitles
resource exhaustion.

=====

119.PERFORMANCE TEST

=====

Measure before/after major optimizations.

Do not optimize based purely on intuition.

=====

120.OPTIMIZATION RULE

=====

MEASURE
→IDENTIFY BOTTLENECK
→CHANGE
→MEASURE AGAIN.

=====

121.NO PREMATURE OPTIMIZATION

=====

Do not optimize code that is not a bottleneck.

=====

122.NO PREMATURE CLOUD

=====

Do not move local workloads to cloud merely because cloud architecture looks sophisticated.

=====

123.NO PREMATURE DATABASE

=====

Use local structured storage for the personal version where appropriate.

=====

124.NO PREMATURE MICROSERVICES

=====

A modular monolith is preferred until scale actually requires separation.

=====

125.REPOSITORY STRUCTURE

=====

Keep clear separation between:

app
components
features
services
domain
adapters
workers
models
storage
utils
tests.

=====

126.DOCUMENTATION

=====

Maintain:

README
architecture
setup
development
testing
limitations
model documentation
provider documentation.

=====

127.DOCUMENTATION TRUTH

=====

Documentation must match implementation.

=====

128.NO STALE DOCUMENTATION

=====

When architecture changes:

update documentation.

=====

129.CONFIGURATION DOCUMENTATION

=====

Document:

required environment variables

optional variables

development setup

production setup.

=====

130.NO CREDENTIAL DOCUMENTATION

=====

Never document actual secret values.

=====

131.LICENSE DOCUMENTATION

=====

Track third-party:

libraries

models

assets

fonts

media.

=====

132.ASSET LICENSING

=====

Do not distribute copyrighted assets without appropriate rights.

=====

133.MODEL LICENSING

=====

Verify:

commercial use
redistribution
training restrictions.

=====

134.MEDIA RIGHTS

=====

The application does not grant rights to content.

Users are responsible for content they supply/use where applicable.

=====

135.PROVIDER COMPLIANCE

=====

Do not design functionality around bypassing provider restrictions.

=====

136.DOWNLOAD FEATURE

=====

A download function may exist only when the source/platform and implementation legitimately permit it.

Do not implement arbitrary content extraction merely because technically possible.

=====

137.NO DRM BYPASS

=====

Never circumvent DRM or access-control systems.

=====

138.NO SECURITY BYPASS

=====

Never disable browser security to force functionality.

=====

139.NO CAPTCHA BYPASS

=====

Never automate around anti-abuse systems.

=====

140.NO RATE-LIMIT BYPASS

=====

Respect provider limits.

=====

141.NO CREDENTIAL HARVESTING

=====

Never request unnecessary third-party credentials.

=====

142.PRIVACY

=====

Local-first.

Minimal collection.

Explicit network behavior.

=====

143.DATA EXPORT

=====

Future feature:

allow users to export local:

history
subscriptions
collections
settings
tickets.

=====

144.DATA IMPORT

=====

Future feature:

restore exported data.

=====

145.SYNC

=====

Cloud synchronization is future.

If introduced:

encrypted
minimal
optional.

=====

146.ACCOUNT SYSTEM

=====

Not required for current personal version.

=====

147.MULTI-DEVICE

=====

Future.

Do not build infrastructure prematurely.

=====

148.MULTI-PROVIDER

=====

Future architecture may support additional providers.

Provider abstraction should make this possible.

=====

149.PLUGIN SYSTEM

=====

Not required now.

Avoid plugin complexity unless justified.

=====

150.FUTURE AI

=====

Potential:

semantic search
personal recommendation
scene understanding
advanced framing
audio personalization
natural-language controls.

All remain optional future layers.

=====

151.FUTURE MODEL TRAINING

=====

Potential custom framing dataset.

Requirements:

authorized data
clear labels
objective metrics
validation
bias/error analysis.

=====

152.FUTURE LOCAL LLM

=====

Potential local LLM for:

search interpretation
media organization
natural-language commands.

It must not become a requirement for playback.

=====

153.FUTURE AUTOMATION

=====

Potential automations:

subscription refresh
metadata refresh
model updates
cleanup
health checks.

All require:

bounded execution
failure handling
resource limits.

=====

154.HEALTH CHECK

=====

Future internal diagnostic:

provider availability
storage health
model availability
browser capability.

=====

155.SELF-DIAGNOSTICS

=====

Diagnostics should report actual state.

Never fabricate "healthy".

=====

156.REPAIR SYSTEM

=====

Future automatic repair may:

clear stale cache
restart worker
reload model
reset temporary state.

It must not silently delete user data.

=====

157.VERSIONING

=====

Application versions should be traceable.

=====

158.RELEASE NOTES

=====

Document meaningful changes.

=====

159.BETA FEATURES

=====

Experimental features must be labeled.

=====

160.EXPERIMENTAL ISOLATION

=====

Experimental code should not destabilize core functionality.

=====

161.FEATURE FLAGS

=====

Use feature flags for risky experiments.

=====

162.FLAG CLEANUP

=====

Remove obsolete flags.

=====

163.BACKWARD COMPATIBILITY

=====
Avoid unnecessary breaking changes.

===== 164.MIGRATION PLAN =====

Breaking changes require:

migration
or
clear reset path.

===== 165.ROLLBACK =====

Maintain a practical rollback strategy for releases.

===== 166.RELEASE CANDIDATE =====

Before release:

freeze feature additions.

===== 167.RELEASE CHECKLIST =====

- [] BUILD
- [] TEST
- [] SECURITY
- [] PERFORMANCE
- [] ACCESSIBILITY
- [] VISUAL
- [] DATA MIGRATION
- [] ERROR HANDLING
- [] DOCUMENTATION
- [] LICENSE REVIEW
- [] REGRESSION.

=====

168.PRODUCTION DECLARATION

=====

Never say:

"100% complete"

unless:

all defined release criteria are actually verified.

=====

169.COMPLETION DEFINITION

=====

COMPLETE means:

specified functionality works,
known critical bugs are resolved,
fallbacks work,
security review passes,
performance is acceptable,
UX is coherent,
documentation matches,
no fake functionality remains.

=====

170.KNOWN LIMITATIONS

=====

A finished product may still have limitations.

Completion does NOT mean:

every browser
every codec
every provider
every device
every model

is supported.

=====

171.TRANSPARENCY

=====

Document limitations rather than hiding them.

=====

172.MAINTENANCE

=====

Periodically review:

dependencies
models
provider behavior
browser APIs
performance
security.

=====

173.DEPENDENCY UPDATE

=====

Never blindly update everything.

Update:

inspect
test
verify.

=====

174.BROWSER API CHANGE

=====

Monitor:

media
fullscreen
WebGPU
filesystem
storage.

=====

175.PROVIDER CHANGE

Provider changes must be isolated through adapters.

176.MODEL CHANGE

Benchmark before replacing.

177.ASSET CHANGE

Verify licensing and performance.

178.PERFORMANCE REGRESSION

Compare against previous stable version.

179.SECURITY REGRESSION

Review newly exposed:

routes
inputs
dependencies
network calls.

180.PRIVACY REGRESSION

Review new:

telemetry
external API
cloud processing
storage.

=====

181.PRODUCT DRIFT

=====

Periodically compare the implementation against:

MASTER GUARDIAN
U-TUBE GUARDIAN
CINEMORPH GUARDIAN
UI/UX GUARDIAN.

=====

182.DRIFT RULE

=====

If implementation starts becoming:

more complicated
more commercial
more cloud-dependent
more ad-like
more social
more cluttered

STOP AND REVIEW.

=====

183.FEATURE CREEP

=====

Move ideas into:

FUTURE
EXPERIMENTAL
REJECTED.

Do not automatically implement.

=====

184.REJECTED IDEAS

=====

Maintain rejected-feature reasoning when useful.

=====

185.DECISION RECORD

=====

For major decisions:

problem
options
decision
reason
tradeoff.

=====

186.AGENT HANDOFF

=====

When another AI/developer takes over:

provide:

architecture
current state
known bugs
limitations
next actions
guardian documents.

=====

187.AGENT RULE

=====

An agent must read relevant guardian documents before major modifications.

=====

188.AGENT INSPECTION

=====

Agent must inspect current implementation rather than assuming previous claims are true.

=====

189.AGENT VERIFICATION

=====

Statements such as:

"already implemented"

"fully working"

"production ready"

must be verified.

=====
190.NO SELF-MERIT
=====

Agents must not praise their own implementation as evidence of quality.

=====
191.NO FAKE COMPLETION
=====

Never claim:

"fully complete"

"100% working"

"production ready"

without evidence.

=====
192.NO FAKE TESTS
=====

Never claim a test passed if it was not executed.

=====
193.NO FAKE BENCHMARK
=====

Never invent performance numbers.

=====
194.NO FAKE SECURITY
=====

Never claim:

"unhackable"
"impossible to attack"
"100% secure."

=====

195.SECURITY LANGUAGE

=====

Use:

"hardened"
"reduced attack surface"
"tested against known threat classes"

where appropriate.

=====

196.NO PERFECT AI

=====

Never claim perfect:

detection
framing
recommendation
classification.

=====

197.NO PERFECT PLAYBACK

=====

Browser/provider/device limitations exist.

=====

198.NO PERFECT COMPATIBILITY

=====

Support must be tested.

=====

199.NO MARKETING OVER ENGINEERING

=====

Do not optimize wording before fixing functionality.

=====

200.PRODUCT REVIEW

=====

Before major release:

review as:

USER
DEVELOPER
SECURITY ENGINEER
PERFORMANCE ENGINEER
QA ENGINEER
ACCESSIBILITY REVIEWER.

=====

201.USER REVIEW

=====

Ask:

Is it understandable?

=====

202.DEVELOPER REVIEW

=====

Ask:

Is it maintainable?

=====

203.SECURITY REVIEW

=====

Ask:

What can be attacked?

=====

204.PERFORMANCE REVIEW

=====

Ask:

What consumes resources?

=====

205.QA REVIEW

=====

Ask:

What breaks?

=====

206.ACCESSIBILITY REVIEW

=====

Ask:

Can different users operate it?

=====

207.PRODUCT REVIEW

=====

Ask:

Does it actually solve the original problem?

=====

208.FINAL SYSTEM TEST

=====

Run the complete journey:

OPEN OMNISTREAM
→CHOOSE U-TUBE
→SEARCH
→WATCH
→NAVIGATE
→MINI PLAYER
→HISTORY
→SUBSCRIBE
→RETURN.

Then:

OPEN OMNISTREAM
→CHOOSE CINEMORPH
→CHOOSE LOCAL FILE
→TICKET
→PREPARE
→THEATER
→1.90
→1.43
→ORIGINAL
→AUDIO
→SUBTITLE
→WATCH
→PAUSE
→EXIT
→RETURN
→TICKET RESUME.

=====
209.FAILURE JOURNEY
=====

Repeat while deliberately causing:

network failure
model failure
audio failure
storage failure
unsupported media
fullscreen failure.

=====
210.SUCCESS CONDITION
=====

The product should remain usable.

=====
211.PERFORMANCE JOURNEY
=====

Repeat with:

3-hour media
large file
high resolution
weak device
multiple tabs.

=====
212.SUCCESS CONDITION
=====

Playback remains prioritized.

=====
213.SECURITY JOURNEY
=====

Test malicious:

URL
metadata
subtitle
filename
media.

=====
214.SUCCESS CONDITION
=====

Application remains isolated and stable.

=====
215.FINAL PRODUCT GUARDIAN
=====

OMNISTREAM MUST ALWAYS BE:

USEFUL
HONEST
FAST ENOUGH
SAFE
PRIVATE
MAINTAINABLE
FREE-FIRST
ACCESSIBLE

RELIABLE.

=====

216.FINAL DEVELOPMENT RULE

=====

DO NOT ASK:

"HOW MANY FEATURES CAN WE ADD?"

ASK:

"WHAT IS THE BEST EXPERIENCE WE CAN BUILD WITH THE RESOURCES WE HAVE?"

=====

217.FINAL ENGINEERING RULE

=====

DO NOT ASK:

"CAN WE TECHNICALLY MAKE IT MORE COMPLEX?"

ASK:

"DOES THE COMPLEXITY CREATE ENOUGH VALUE TO JUSTIFY IT?"

=====

218.FINAL AI RULE

=====

DO NOT ASK:

"WHERE CAN WE ADD AI?"

ASK:

"WHERE DOES INTELLIGENCE PRODUCE A MEASURABLE USER BENEFIT?"

=====

219.FINAL UX RULE

=====

DO NOT ASK:

"HOW MANY UI ELEMENTS CAN WE SHOW?"

ASK:

"WHAT DOES THE USER NEED AT THIS EXACT MOMENT?"

=====
220.FINAL PERFORMANCE RULE
=====

DO NOT ASK:

"CAN THE DEVICE RUN THE MODEL?"

ASK:

"CAN THE DEVICE RUN THE MODEL WHILE THE USER STILL HAS A SMOOTH
EXPERIENCE?"

=====
221.FINAL SECURITY RULE
=====

DO NOT ASK:

"IS IT SECURE?"

ASK:

"WHAT CAN FAIL,WHAT CAN BE ATTACKED,WHAT CAN LEAK,AND HOW DO WE CONTAIN
IT?"

=====
222.FINAL RELEASE RULE
=====

DO NOT ASK:

"DOES IT LOOK FINISHED?"

ASK:

"CAN A REAL USER USE IT RELIABLY WITHOUT OUR HELP?"

=====

223.FINAL GUARDIAN CHECK

=====

BEFORE RELEASE:

READ THE FIVE GUARDIAN DOCUMENTS.

COMPARE IMPLEMENTATION AGAINST THEM.

IDENTIFY DRIFT.

FIX CRITICAL DRIFT.

DOCUMENT ACCEPTED DEVIATIONS.

TEST.

RETEST.

ONLY THEN RELEASE.

=====

224.FINAL OMNISTREAM PRINCIPLE

=====

OMNISTREAM SHOULD NOT TRY TO WIN BY BEING THE BIGGEST.

IT SHOULD WIN BY BEING:

SIMPLER THAN THE COMPLEX ALTERNATIVE,
MORE PERSONAL THAN THE GENERIC PLATFORM,
MORE IMMERSIVE WHEN ENTERING CINEMORPH,
MORE HONEST ABOUT ITS LIMITATIONS,
MORE PRIVATE WHERE POSSIBLE,
AND MORE RELIABLE THAN ITS FEATURE COUNT SUGGESTS.

THE USER SHOULD FEEL:

U-TUBE:

"I CAN FIND AND WATCH WHAT I WANT."

CINEMORPH:

"I HAVE ENTERED A CINEMA."

THE ENGINEERING SHOULD MAKE BOTH STATEMENTS TRUE.

[END_OMNISTREAM_DOCUMENT_05]